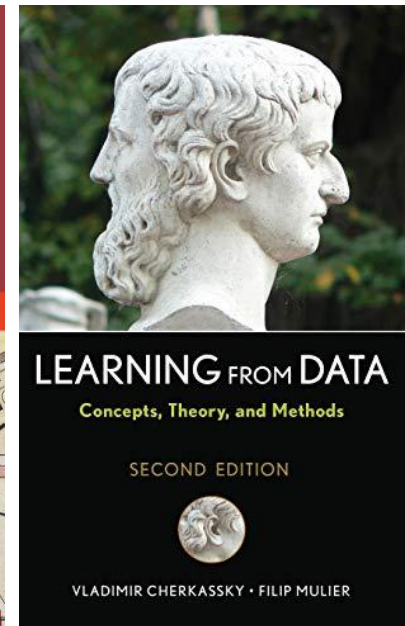
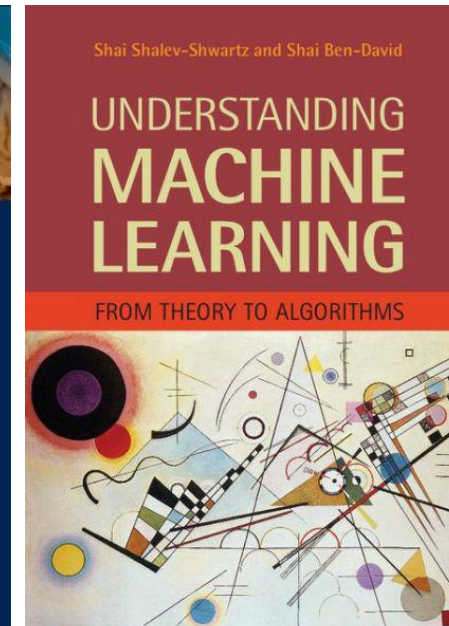
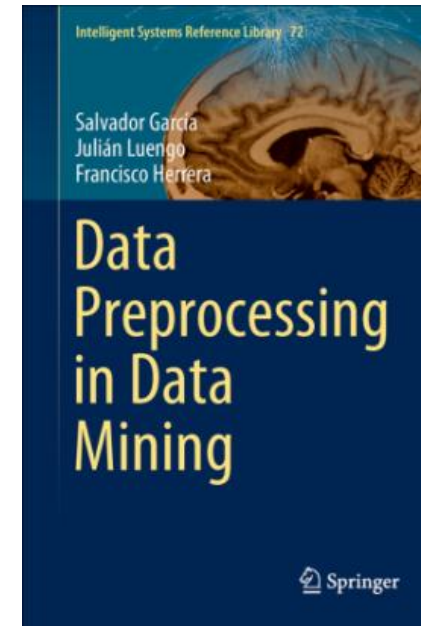
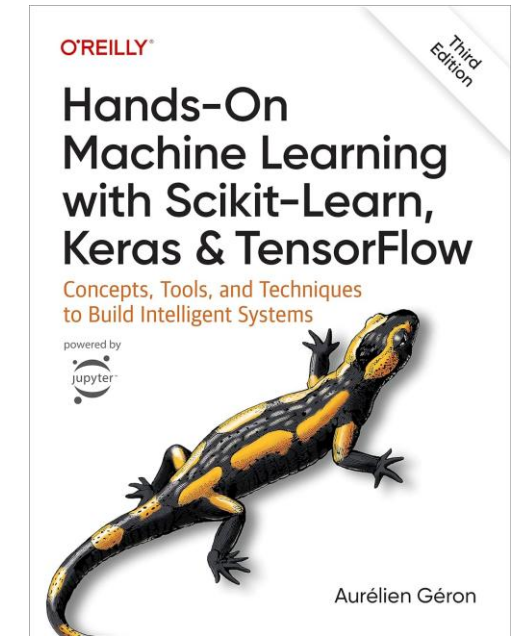


Aprendizaje Automático

Curso 2025-2026

Tema 3. Calidad de datos. Parte II

Bibliografía



- 
- Tema 1: Conceptos básicos del Aprendizaje Automático
 - Tema 2: Aprendizaje Supervisado. Regresión y Clasificación. Modelos. Evaluación
 - **Tema 3: Ingeniería de Características y Preprocesamiento de Datos. Calidad de Datos**
 - Tema 4: Aprendizaje no Supervisado
 - Tema 5: Aprendizaje Profundo. Fundamentos
 - Tema 6: Aprendizaje Profundo en diferentes tipos de datos y problemas
 - Tema 7. Aprendizaje por refuerzo
 - Tema 8: Optimización en Aprendizaje Automático. Hiperparámetros, selección de modelos y regularización
 - Tema 9: Aspectos Avanzados, retos y desafíos del aprendizaje automático



Calidad de Datos

Calidad de datos

Introducción al preprocesamiento de datos

Integración, limpieza y transformación

Datos imperfectos

Reducción de datos

Discretización

Selección de Características

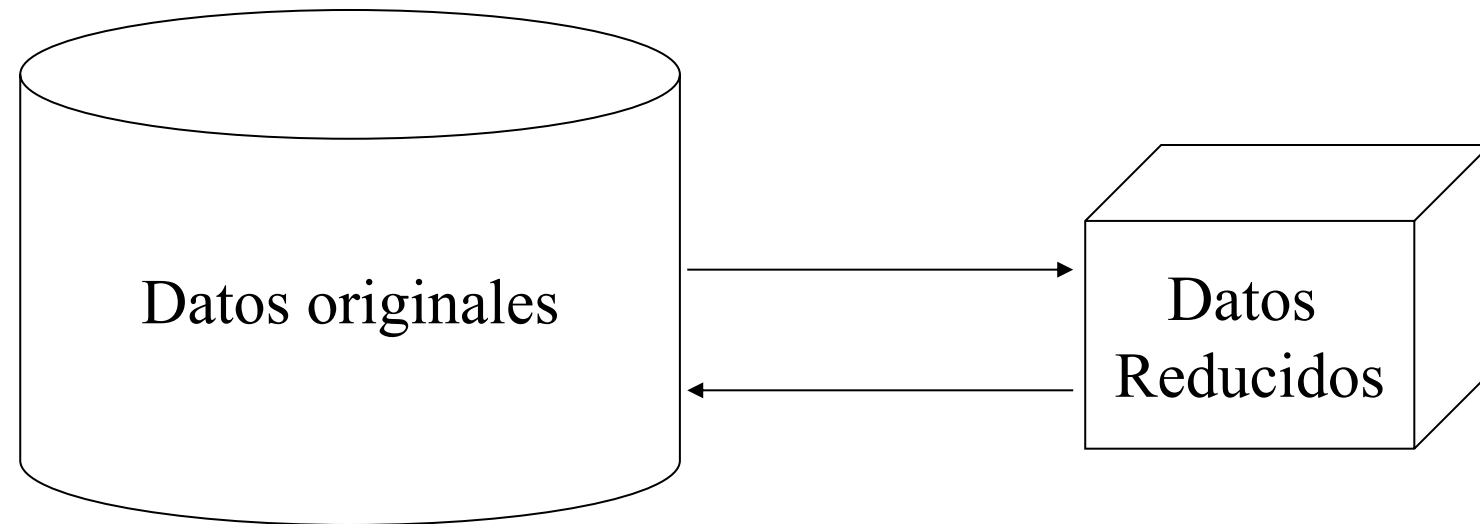
Selección de Instancias

Comentarios finales

Suplemento: Reducción de Dimensionalidad

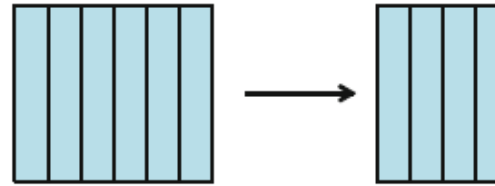
Reducción de datos

- ❖ Selecciona/extrae datos relevantes para la tarea de la minería de datos/extracción de información.

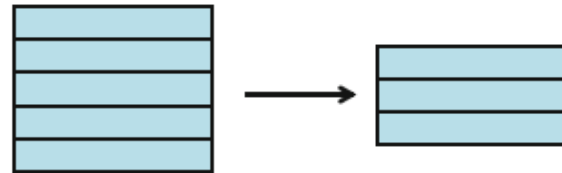


Reducción de datos

Feature Selection



Instance Selection



Discretization

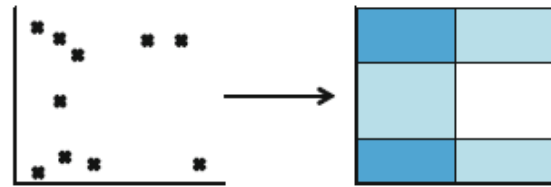


Fig. 1.4 Forms of data reduction



Calidad de Datos

Calidad de datos

Introducción al preprocesamiento de datos

Integración, limpieza y transformación

Datos imperfectos

Reducción de datos

Discretización

Selección de Características

Selección de Instancias

Comentarios finales

Suplemento: Reducción de Dimensionalidad



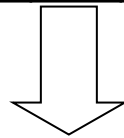
Discretización

- Los valores discretos son muy útiles en aprendizaje automático
- Representan información más concisa, son más fáciles de entender más cercanos a la representación a nivel de conocimiento.
- La discretización busca transformar los valores continuos/discretos que se encuentran ordenados en valores nominales que no están ordenados. Proceso de cuantificación de atributos numéricos.
- Los valores nominales tienen un dominio finito, por lo que también se considera una técnica de reducción de datos.
- La discretización puede hacerse antes de la obtención de conocimiento o durante la etapa de obtención de conocimiento.

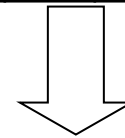
Discretización

- Divide el rango de atributos continuos (numéricos) en intervalos
- Almacena solo las etiquetas de los intervalos
- Importante para reglas de asociación y clasificación, algunos algoritmos solo aceptan datos discretos.

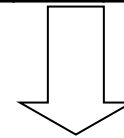
EDAD	5	6	6	9	...	15	16	16	17	20	...	24	25	41	50	65	...	67
COCHE EN PROPIEDAD	0	0	0	0	...	0	1	0	1	1	...	0	1	1	1	1	...	1



EDAD [5,15]

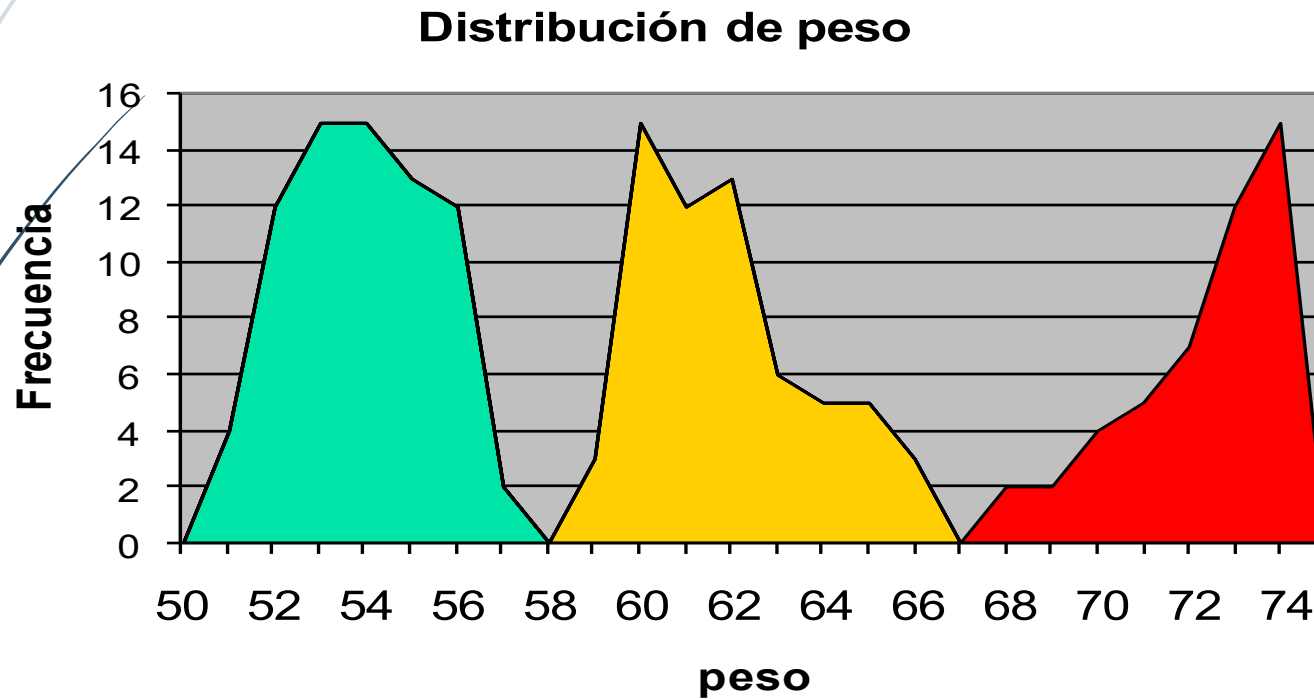


EDAD [16,24]



EDAD [25,67]

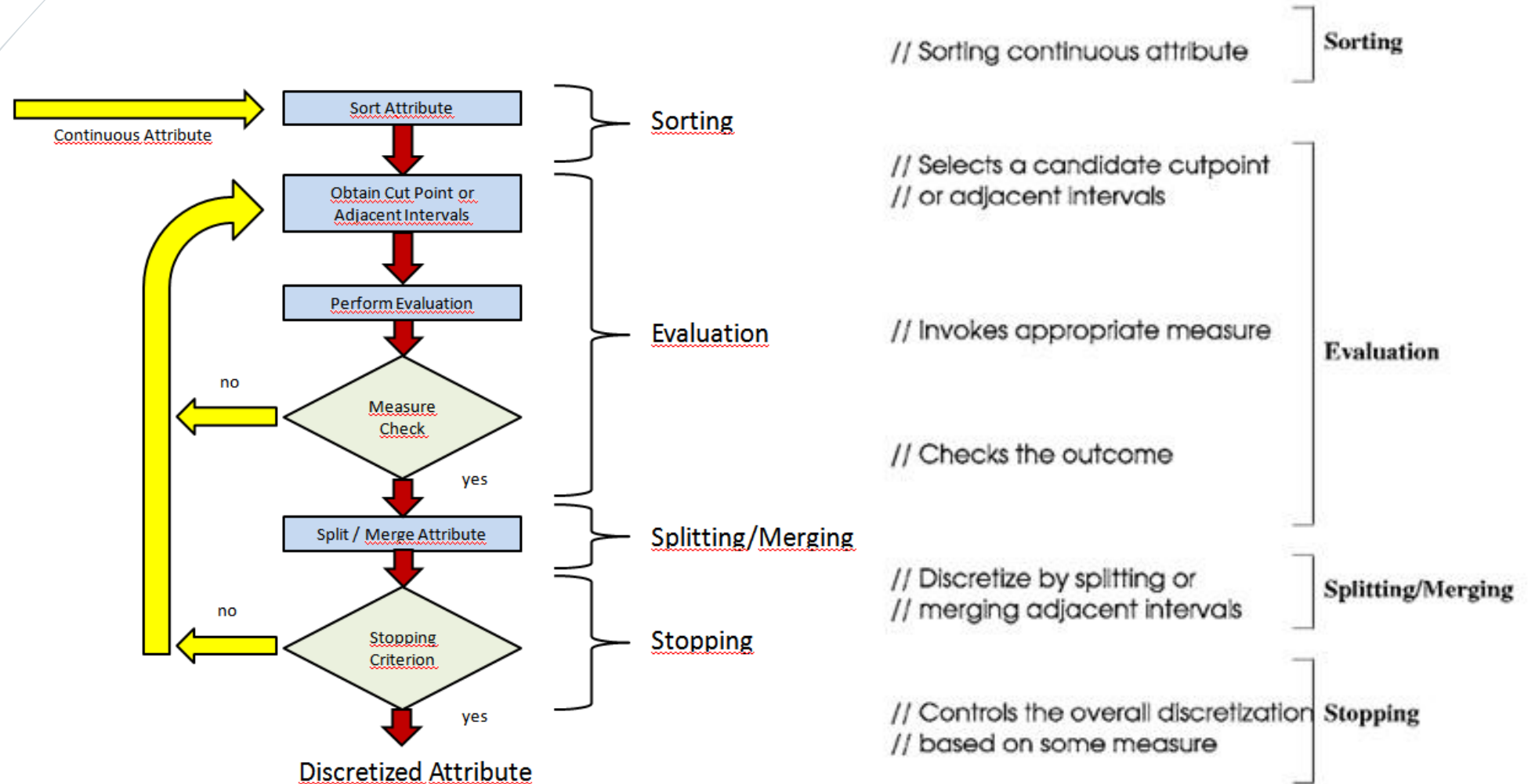
Discretización



50 - 58 kg
59-67 kg
> 68 kg

Discretización

Etapas en el proceso de discretización



Discretización

- **La discretización se ha desarrollado a lo largo de diferentes líneas según las necesidades:**
- **Supervisados vs. No supervisados:** Consideran o no el atributo objetivo.
- **Dinámicos vs. estáticos:** Mientras se construye o no el modelo.
- **Locales vs. Globales:** Centrados en una subregión del espacio de instancias o considerando todas ellas.
- **Top-down vs. Bottom-up:** Empiezan con una lista vacía o llena de puntos de corte.
- **Directos vs. Incrementales:** Usan o no un proceso de optimización posterior.

Discretización

➤ Algoritmos no supervisados:

- Intervalos de igual amplitud
- Intervalos de igual frecuencia
- Clustering

➤ Algoritmos supervisados:

➤ Basados en Entropía [Fayyad & Irani 93 and others]

[Fayyad & Irani 93] U.M. Fayyad and K.B. Irani. Multi-interval discretization of continuous-valued attributes for classification learning. *Proc. 13th Int. Joint Conf. AI (IJCAI-93)*, 1022-1027. Chamberry, France, Aug./ Sep. 1993.

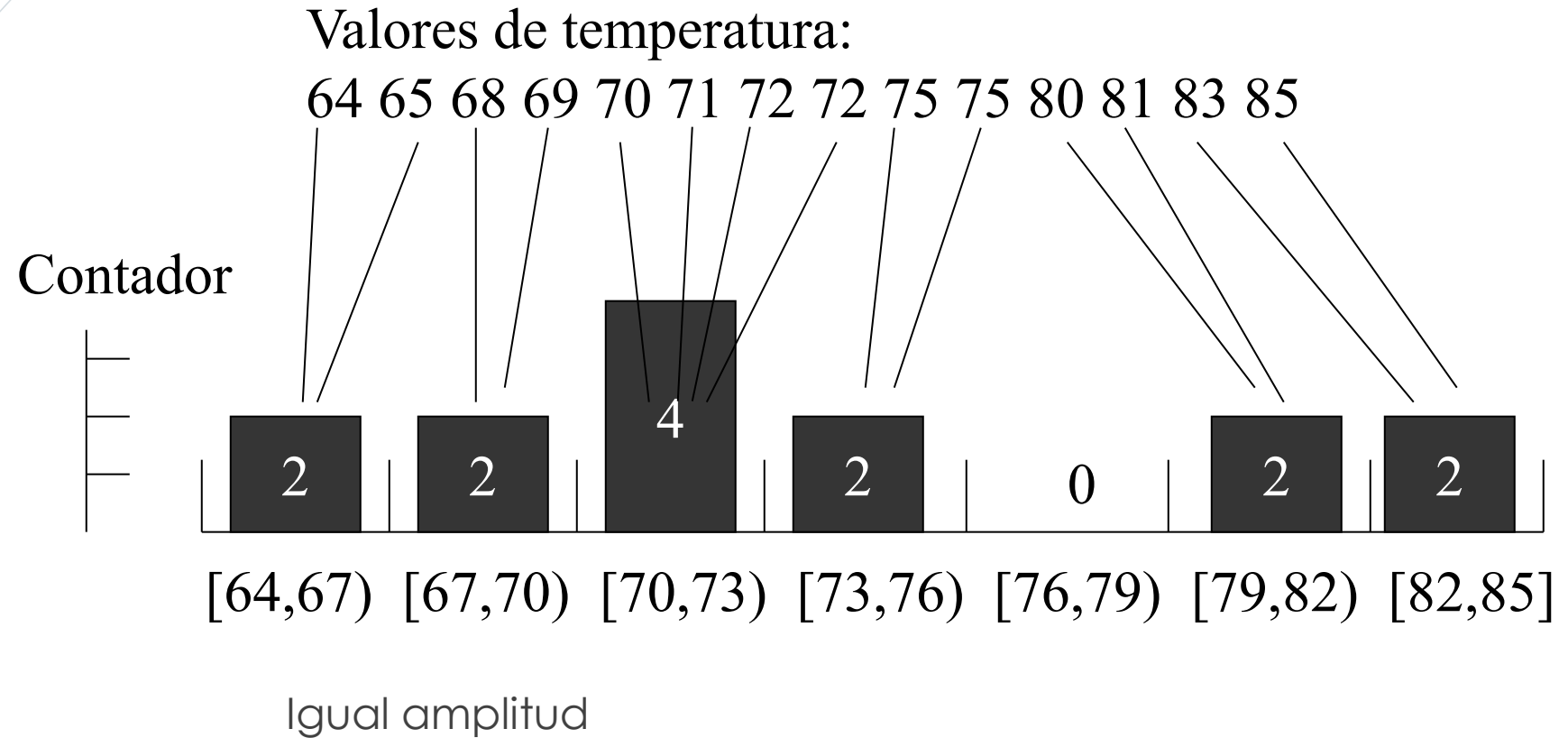
➤ Metodos Chi-square [Kerber 92]

[Kerber 92] R. Kerber. ChiMerge: Discretization of numeric attributes. *Proc. 10th Nat. Conf. AAAI*, 123-128. 1992.

➤ ... (múltiples propuestas)

Bibliografía: S. García, J. Luengo, José A. Sáez, V. López, F. Herrera, A Survey of Discretization Techniques: Taxonomy and Empirical Analysis in Supervised Learning. *IEEE Transactions on Knowledge and Data Engineering* 25:4 (2013) 734-750, [doi: 10.1109/TKDE.2012.35](https://doi.org/10.1109/TKDE.2012.35).

Discretización



Discretización

Discretización:
igual amplitud puede producir desequilibrios

Contador



Discretización

Problemas con igual amplitud

Contador



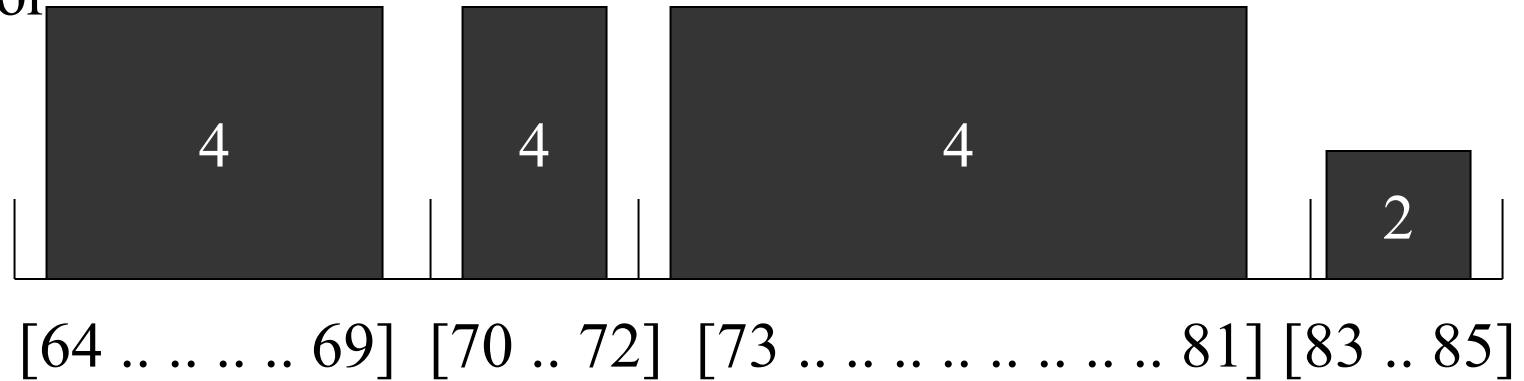
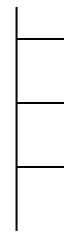
¿Qué podemos hacer para conseguir una mejor distribución?

Discretización

Valores de la temperatura

64 65 68 69 70 71 72 72 75 75 80 81 83 85

Contador



Igual frecuencia (altura) = 4, excepto para la última caja

Discretización

- Algoritmos no supervisados:

- Intervalos de igual amplitud
- Intervalos de igual frecuencia
- Clustering

- Algoritmos supervisados:

- Basados en Entropía [Fayyad & Irani 93 and others]

[Fayyad & Irani 93] U.M. Fayyad and K.B. Irani. Multi-interval discretization of continuous-valued attributes for classification learning. *Proc. 13th Int. Joint Conf. AI (IJCAI-93)*, 1022-1027. Chamberry, France, Aug./ Sep. 1993.

- Metodos Chi-square [Kerber 92]

[Kerber 92] R. Kerber. ChiMerge: Discretization of numeric attributes. *Proc. 10th Nat. Conf. AAAI*, 123-128. 1992.

- ... (múltiples propuestas)

Bibliografía: S. García, J. Luengo, José A. Sáez, V. López, F. Herrera, A Survey of Discretization Techniques: Taxonomy and Empirical Analysis in Supervised Learning. *IEEE Transactions on Knowledge and Data Engineering* 25:4 (2013) 734-750, [doi: 10.1109/TKDE.2012.35](https://doi.org/10.1109/TKDE.2012.35).

Discretización

Discretizador Entropy MDLP (Fayyad)

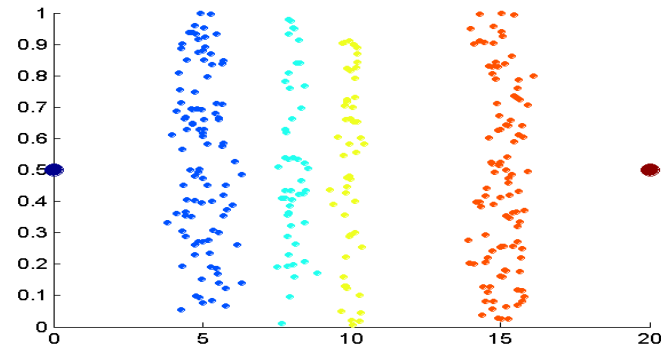
- Comienzan con los puntos de corte dados entre ejemplos de **diferentes** clases:

11	14	15	18	19	20	21	22	23	25	30	31	33	35	36
R	C	C	R	C	R	C	R	C	C	R	C	R	C	R

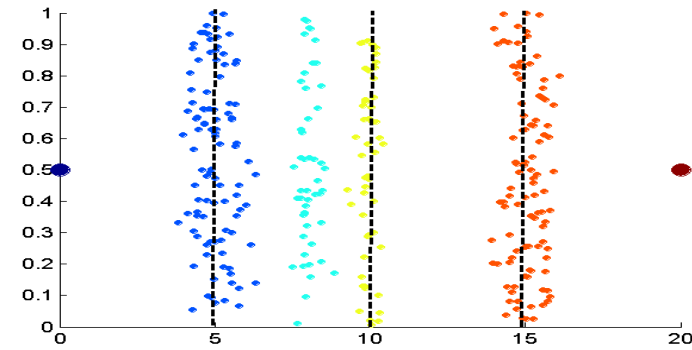
- Minimum Description Length Principle (MDLP), basado en entropía, se utiliza para escoger los puntos de corte útiles entre los anteriores.
- El criterio de parada se basa también en MDLP.
- MDLP se formula como el problema de encontrar el coste de comunicación entre un emisor y un receptor. Se asume que el emisor tiene el conjunto de instancias mientras que el receptor tiene las etiquetas de clase.
- Se dice que una partición inducida por un punto de corte es aceptada si y solo si el coste del mensaje requerido para enviar antes de particionar es mayor que el requerido después de particionar.

Discretización

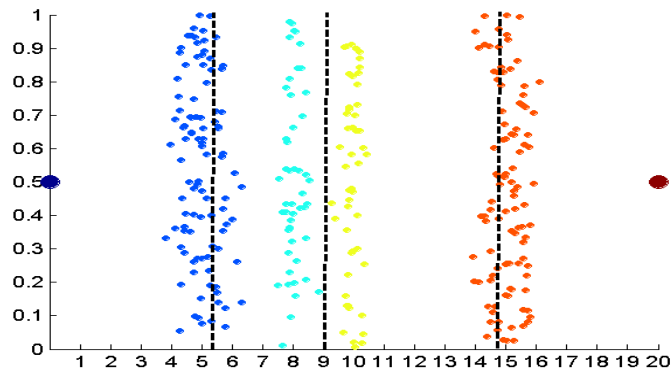
Discretización sin utilizar las clases



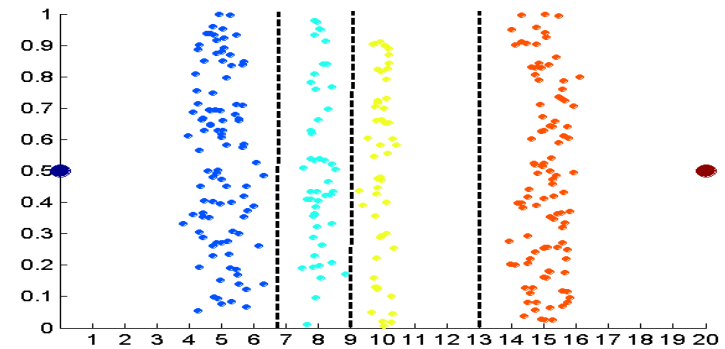
Datos



Igual anchura de intervalo



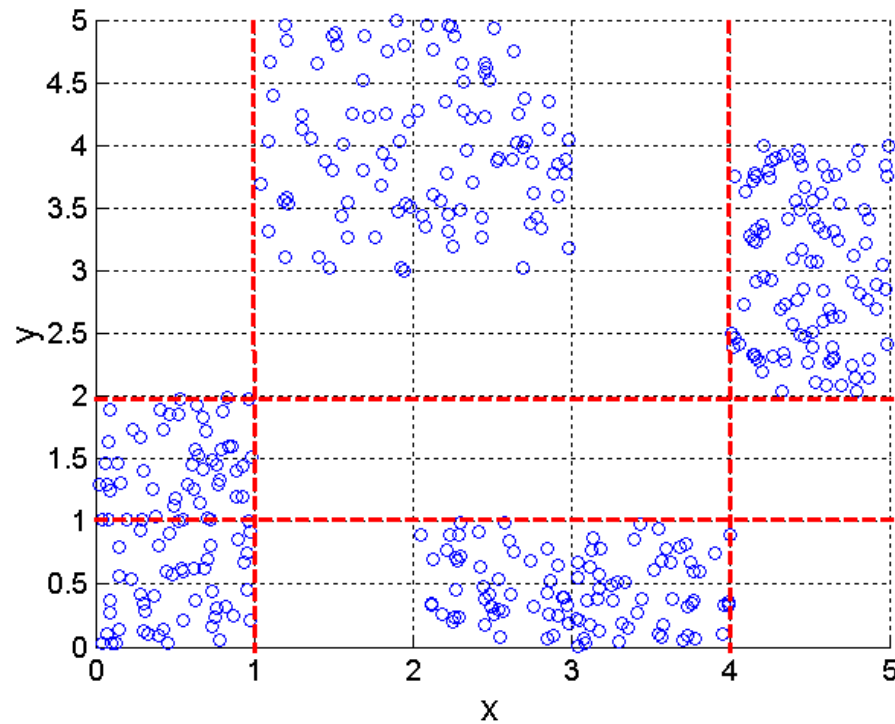
Igual frecuencia



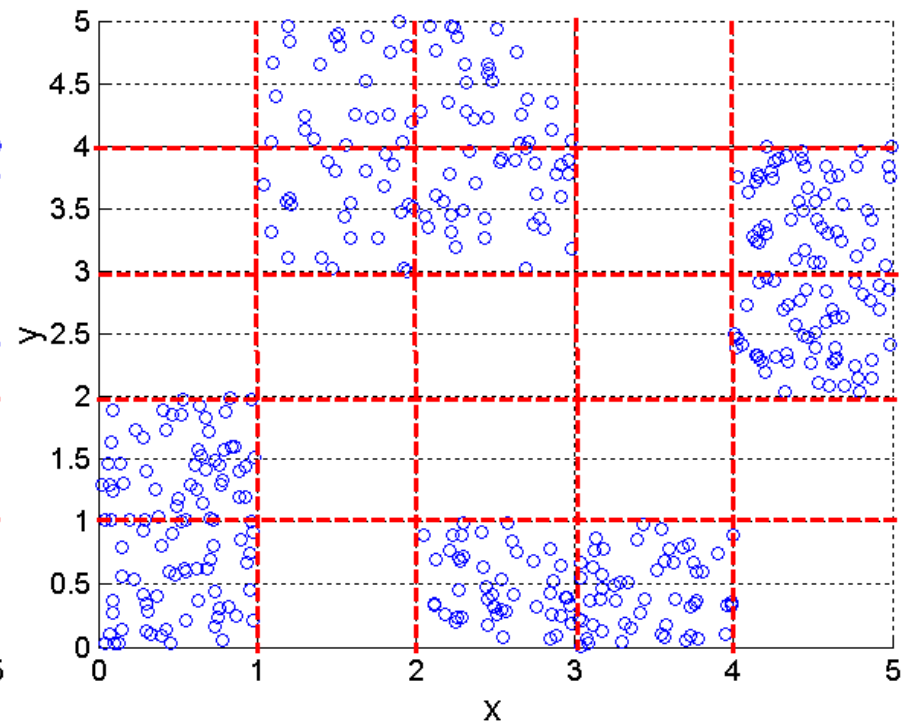
K-medias

Discretización

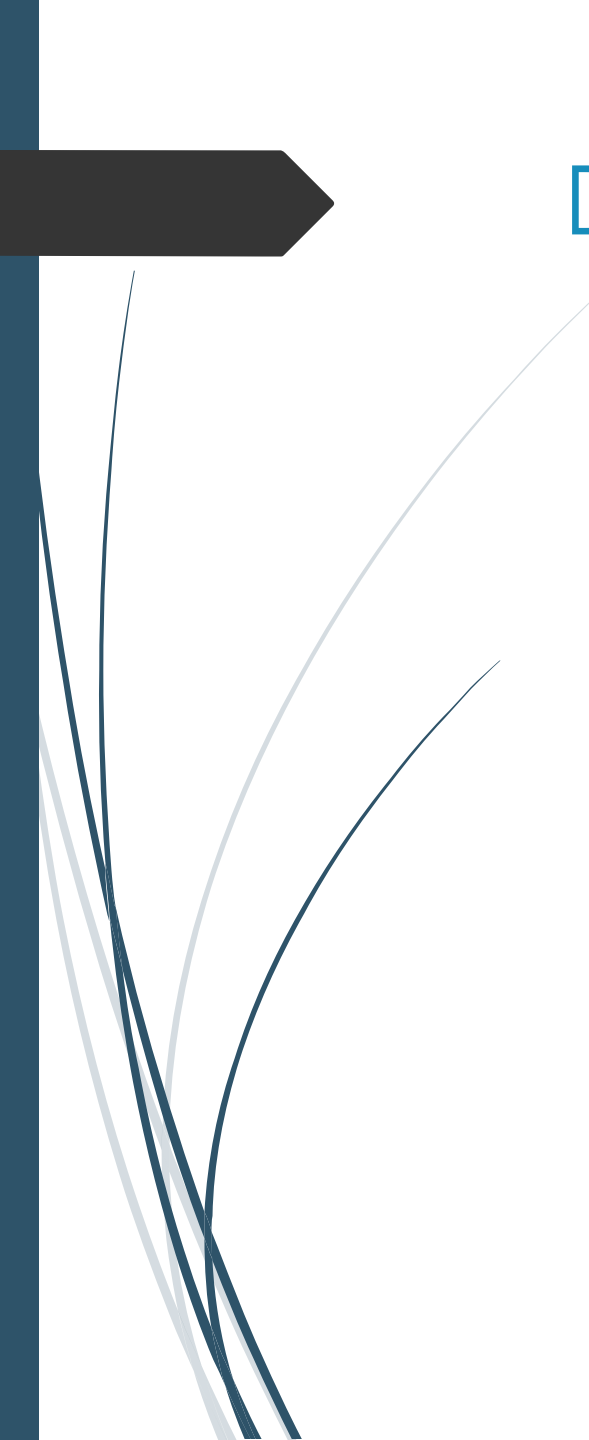
Discretización utilizando clases (basado en entropía)



3 categorías para ambas variables x e y



5 Categorías para ambas variables



Discretización

- *¿Qué discretizador será mejor?*
- Como siempre, dependerá de la aplicación, necesidades del usuario, etc...
- Formas de evaluación:
 - Número total de intervalos
 - Número de inconsistencias causadas
 - Tasa de acierto predictivo



Calidad de Datos

Calidad de datos

Introducción al preprocesamiento de datos

Integración, limpieza y transformación

Datos imperfectos

Reducción de datos

Discretización

Selección de Características

Selección de Instancias

Comentarios finales

Suplemento: Reducción de Dimensionalidad

Selección de Características

El problema de la selección de características (SC) o variables (*Feature Subset Selection, FSS*) consiste en encontrar un subconjunto de las variables del problema que optimice la probabilidad de clasificar correctamente

¿Por qué es necesaria la selección de variables?

- Más atributos no significa más éxito en la clasificación
- Trabajar con menos variables reduce la complejidad del problema y disminuye el tiempo de ejecución
- Con menos variables la capacidad de generalización aumenta
- Los valores para ciertos atributos pueden ser costosos de obtener

Selección de Características

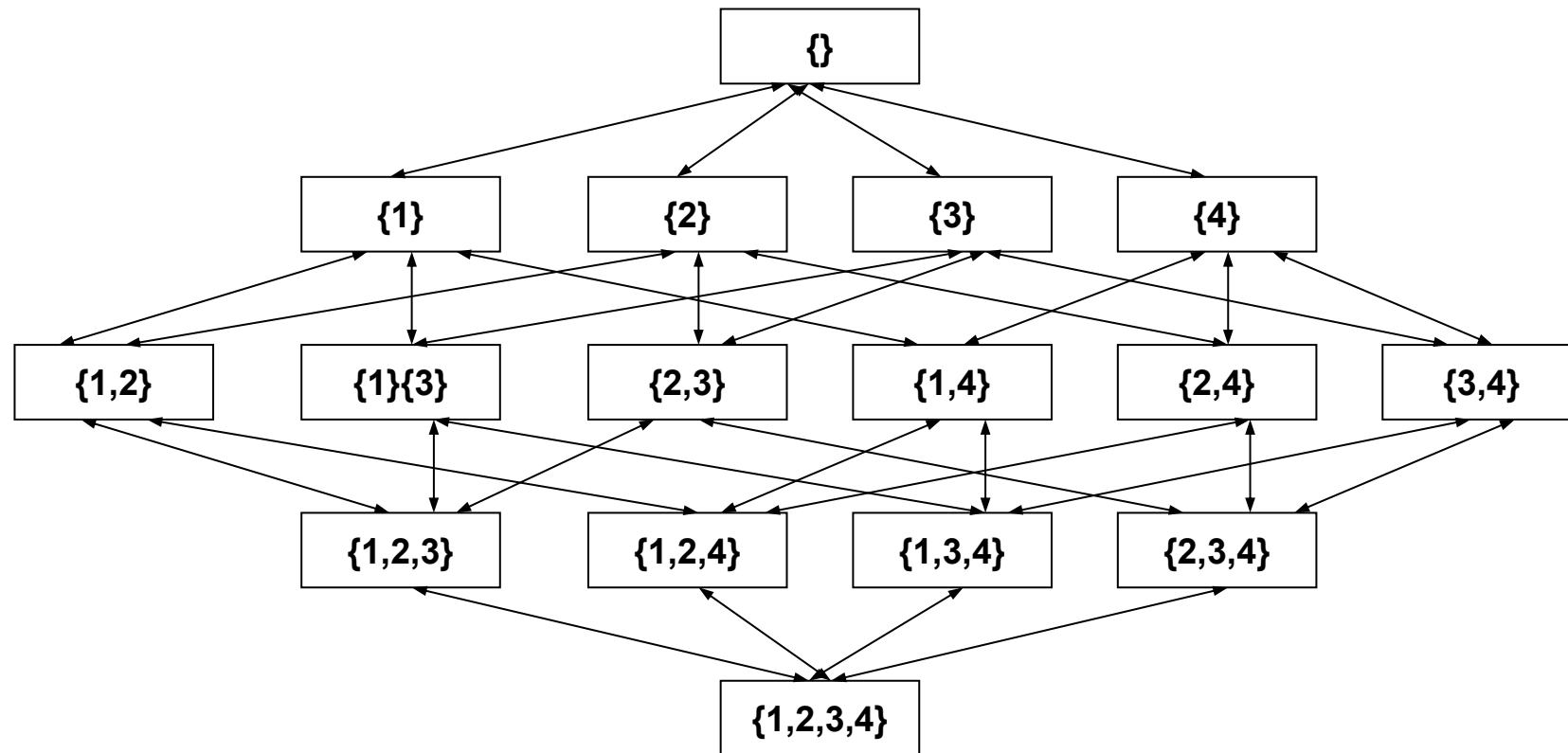
✿ El resultado de la SC sería:

- ❖ Menos datos → los algoritmos pueden aprender más rápidamente
- ❖ Mayor exactitud → el clasificador generaliza mejor
- ❖ Resultados más simples → más fácil de entender

✿ SC tiene su extensión en la Transformación (extracción y construcción de atributos)

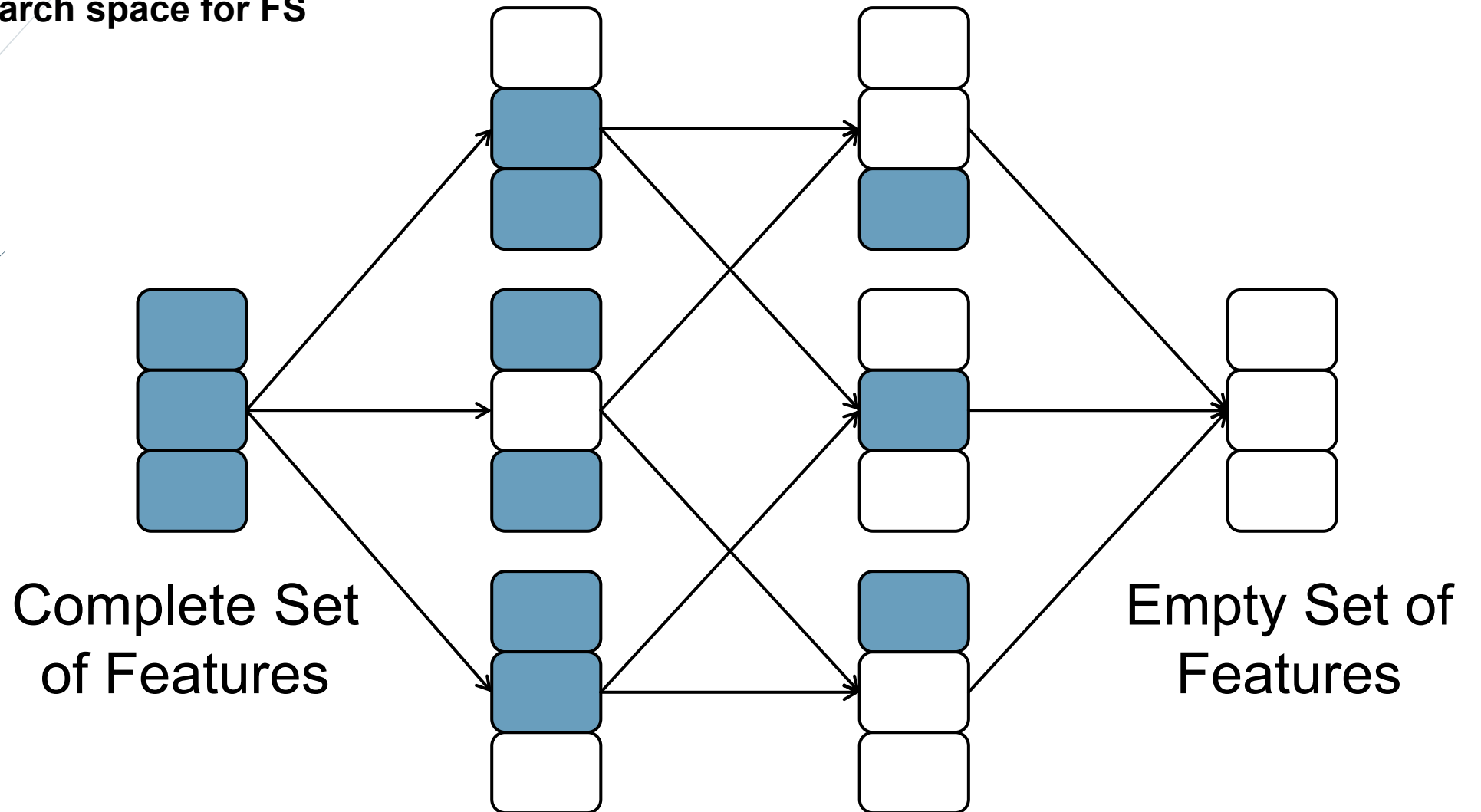
Selección de Características

La SC se puede considerar como un problema de búsqueda



Selección de Características

Fig. 7.1 Search space for FS



Selección de Características

- En un algoritmo de selección de características se distinguen dos componentes principales
 - Una estrategia de búsqueda para seleccionar subconjuntos candidatos
 - Una función objetivo que evalúe esos subconjuntos
- Estrategia de búsqueda
 - Dadas N variables, explorar todos los subconjuntos posibles supone 2^N (p.e. $2^{20}=1048576$)
 - Si queremos exactamente subconjuntos de M variables ($M \leq N$) entonces supone $\binom{N}{M}$. P.e. explorar subconjuntos de 10 variables de 20 posibles, daría 184756
 - Una búsqueda exhaustiva no es aceptable
- Función objetivo: evaluar la bondad del subconjunto seleccionado

Selección de Características

Ejemplo: Espacio de atributos para el problema *weather*

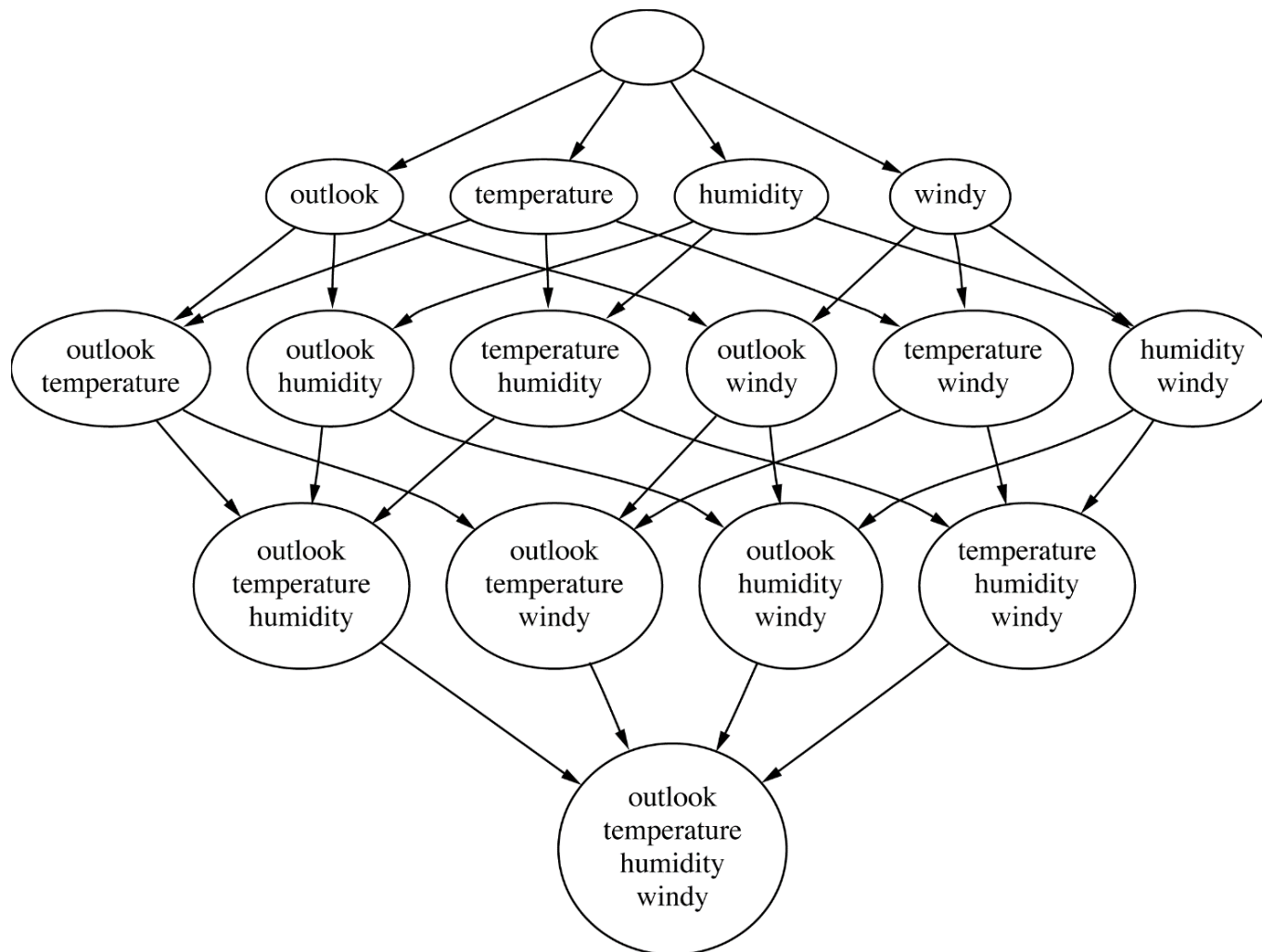
Algorithm 1 Sequential forward feature set generation - SFG.

```
function SFG( $F$  - full set,  $U$  - measure)
  initialize:  $S = \{\}$                                       $\triangleright S$  stores the selected features
  repeat
     $f = \text{FINDNEXT}(F)$ 
     $S = S \cup \{f\}$ 
     $F = F - \{f\}$ 
  until  $S$  satisfies  $U$  or  $F = \{\}$ 
  return  $S$ 
end function
```

Algorithm 2 Sequential backward feature set generation - SBG.

```
function SBG( $F$  - full set,  $U$  - measure)
  initialize:  $S = \{\}$                                       $\triangleright S$  holds the removed features
  repeat
     $f = \text{GETNEXT}(F)$ 
     $F = F - \{f\}$ 
     $S = S \cup \{f\}$ 
  until  $S$  does not satisfy  $U$  or  $F = \{\}$ 
  return  $F \cup \{f\}$ 
end function
```

Ejemplo: Espacio de atributos para el problema *weather*



Selección de Características

Funciones objetivo: Se distinguen dos enfoques distintos

- **Filtro (*filter*)**. La función objetivo evalúa los subconjuntos basándose en la información que contienen. Se utiliza como función objetivo medidas de separabilidad de clases, de dependencias estadísticas, basadas en teoría de la información,...)
- **Envolvente (*wrapper*)**. La función objetivo consiste en aplicar la técnica de aprendizaje que se utilizará finalmente sobre la proyección de los datos al conjunto de variables candidato. El valor devuelto suele ser el porcentaje de acierto del clasificador construido

Selección de Características

Medidas filtro

- **Medidas de separabilidad.** Miden la separabilidad entre clases: euclídeas, Mahalanobis, ...
 - P.e. Para un problema con 2 clases, un proceso de SC basado en medidas de este tipo determina que X es mejor que Y si X induce una diferencia mayor que Y entre las dos probabilidades condicionales de las clases
- **Correlaciones.** Serán buenos subconjuntos los que estén muy correlacionados con la clase

$$f(X_1, \dots, X_M) = \frac{\sum_{i=1}^M \rho_{ic}}{\sum_{i=1}^M \sum_{j=i+1}^M \rho_{ij}}$$

donde ρ_{ic} es el coeficiente de correlación entre la variable X_i y la etiqueta c de la clase (C) y ρ_{ij} es el coeficiente de correlación entre X_i y X_j

La correlación sólo puede medir dependencias lineales.

Selección de Características

■ Medidas basadas en teoría de información

- Un método bastante más potente es la información mutua $I(X_{1,...,M}; C)$

$$f(X_{1,...,M}) = I(X_{1,...,M}; C) = H(C) - H(C|X_{1,...,M}) =$$
$$\sum_{c=1}^{|C|} \int_{X_{1,...,M}} P(X_{1,...,M}, \omega_c) \log \frac{P(X_{1,...,M}, \omega_c)}{P(X_{1,...,M})P(\omega_c)} dx$$

donde H representa la entropía y ω_c la c-ésima etiqueta de la clase C

- La información mutua mide la cantidad de incertidumbre que disminuye en la clase C cuando se conocen los valores del vector $X_{1,...,M}$
- Por la complejidad de cálculo de I se suelen utilizar reglas heurísticas

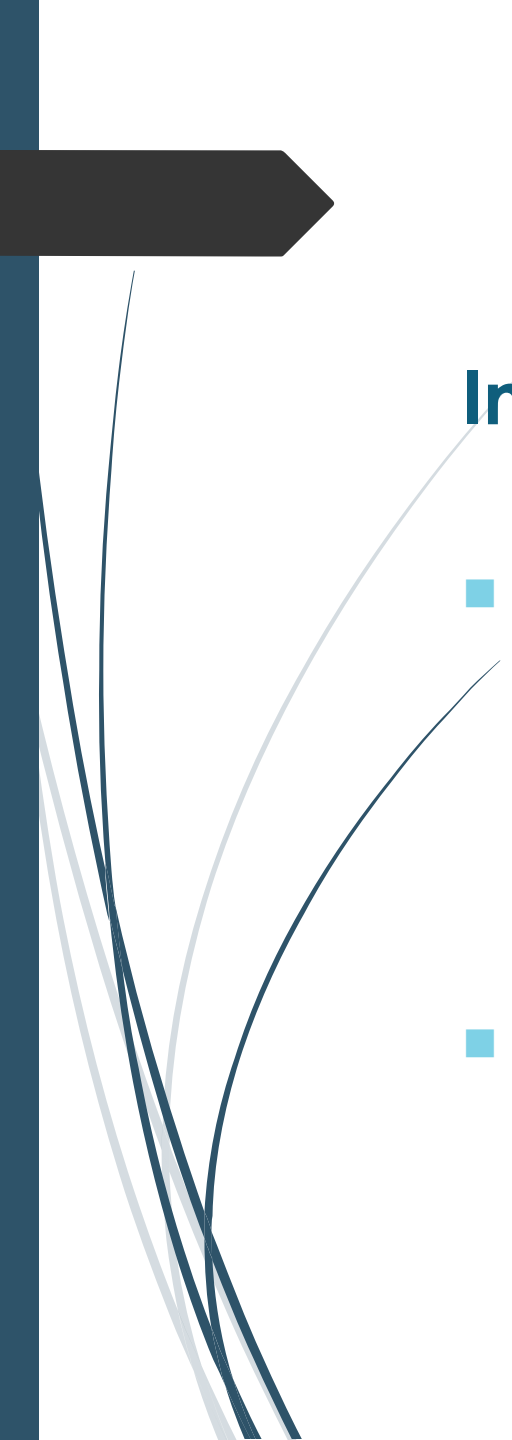
por ejemplo con $\beta=0.5$

$$f(X_{1,...,M}) = \sum_{i=1}^M I(X_i; C) - \beta \sum_{i=1}^M \sum_{j=i+1}^M I(X_i; X_j)$$

Selección de Características

■ Medidas de consistencia

- Los tres grupos de medidas anteriores intentan encontrar las características que puedan, de forma maximal, predecir una clase frente al resto
 - Este enfoque no puede distinguir entre dos variables igualmente adecuadas, no detecta variables redundantes
- Las medidas de consistencia tratan de encontrar el mínimo número de características que puedan separar las clases de la misma forma que lo hace el conjunto completo de variables



Selección de Características

Inconvenientes

■ Envolvertes:

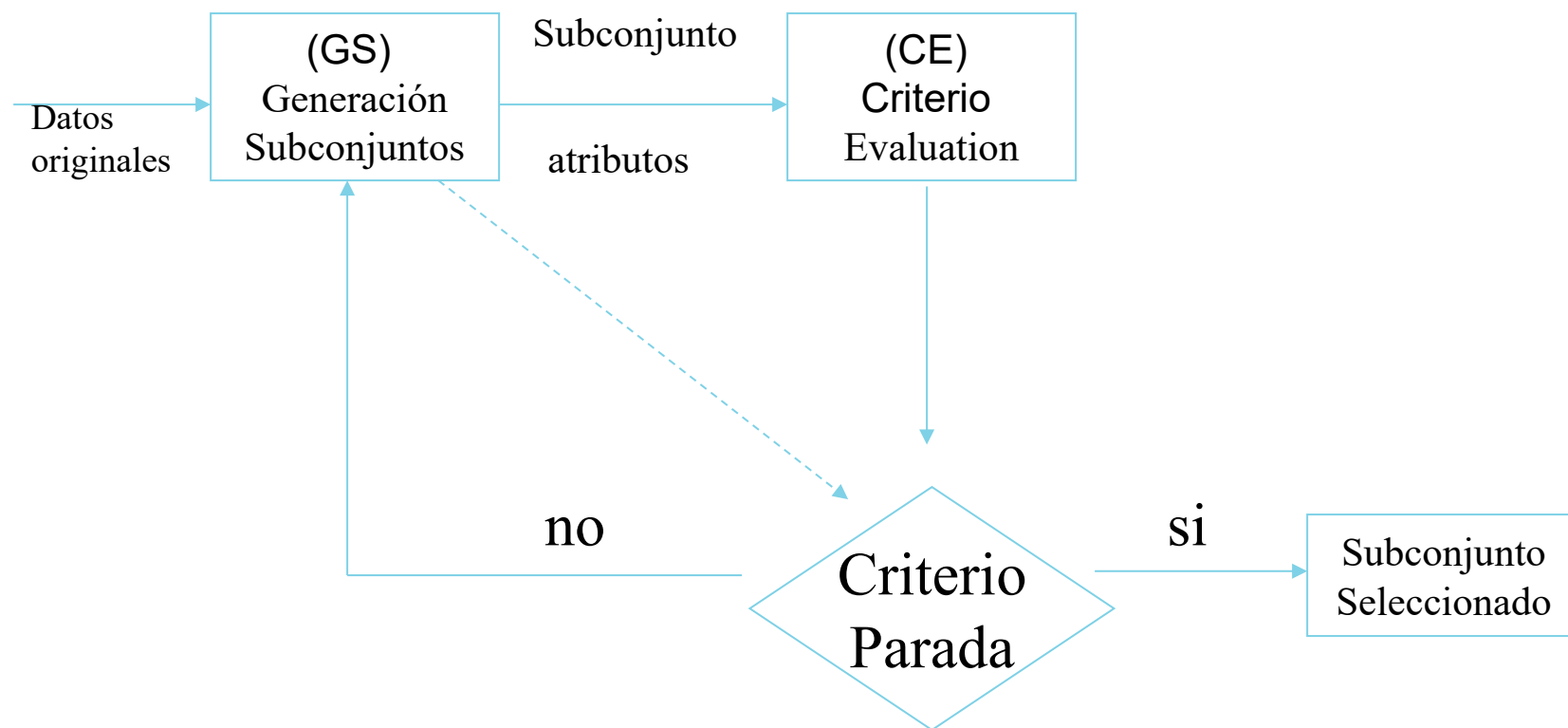
- Muy costosos: para cada evaluación hay que aprender un modelo y validarlo. No es factible para clasificadores costosos
- Pérdida de generalidad: La solución está sesgada hacia el clasificador utilizado

■ Filtros:

- Tendencia a incluir muchas variables. Normalmente se debe a las características monótonas de la función objetivo utilizada
 - El usuario deberá seleccionar un umbral

Selección de Características

Proceso



Selección de Características

Proceso

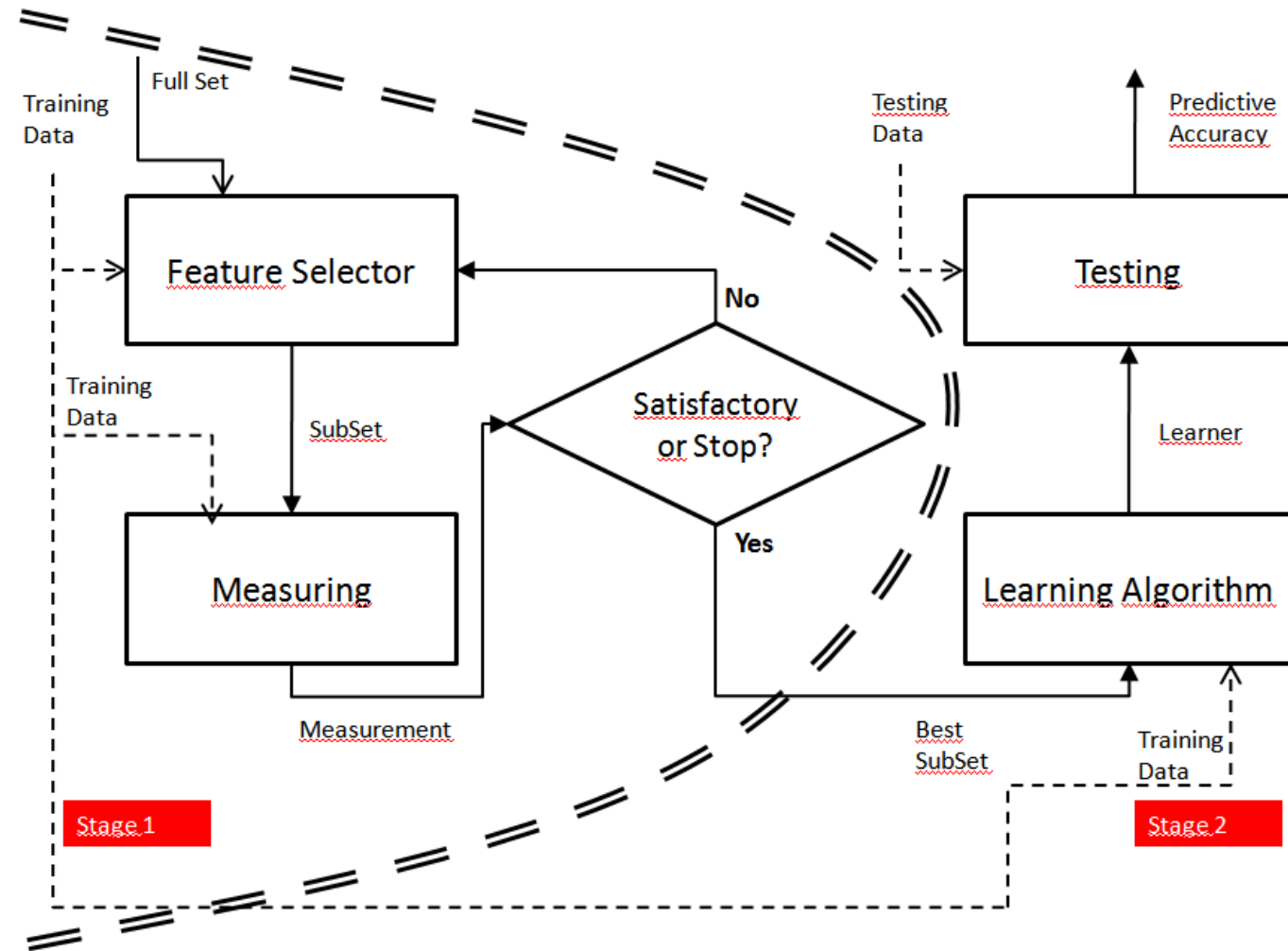


Fig. 7.2 A filter model for FS

Selección de Características

Proceso

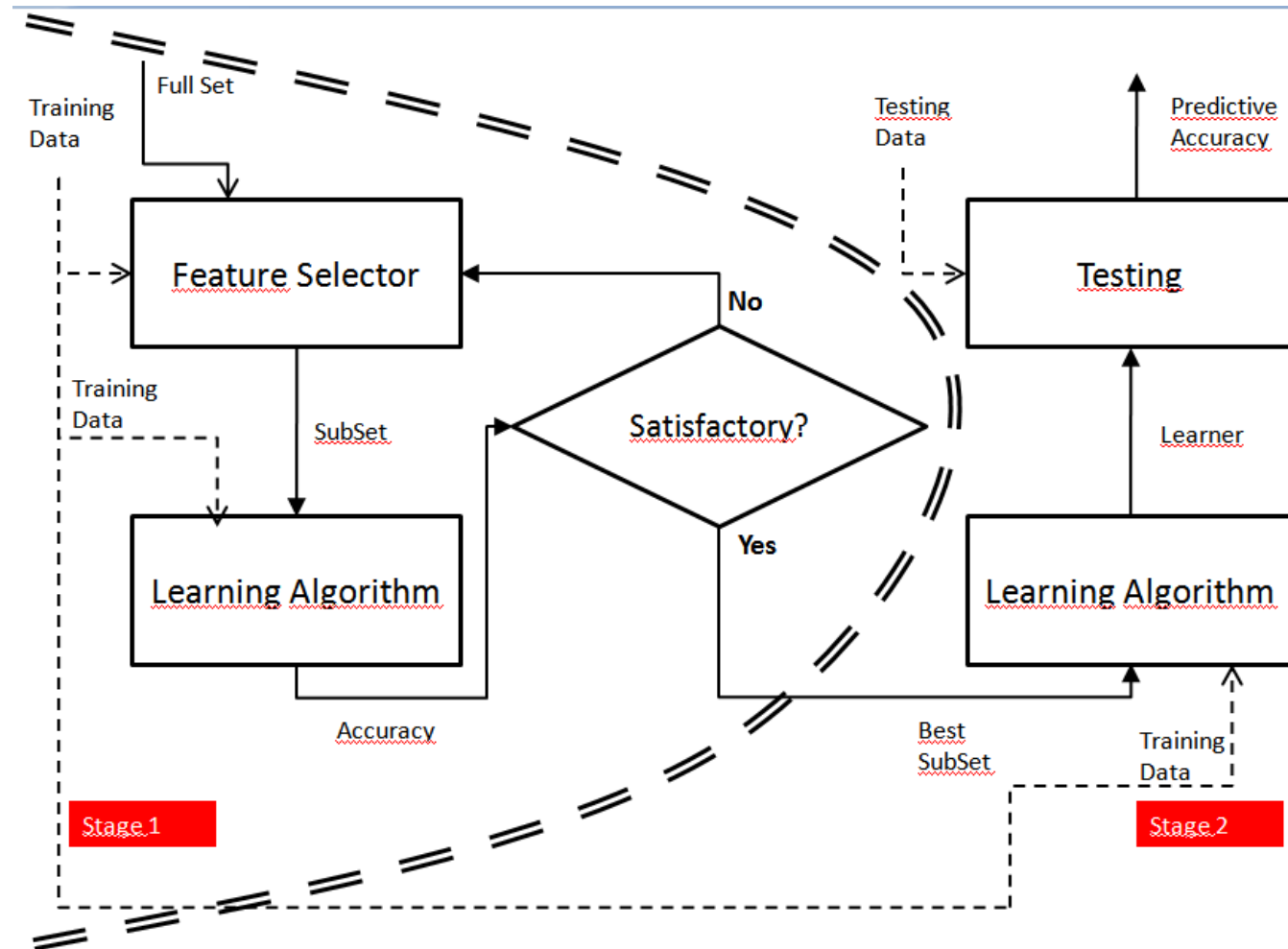


Fig. 7.2 A wrapper model for FS

Selección de Características

Distintas Clasificaciones

1. Según la evaluación:

filter

wrapper

2. Disponibilidad de la clase:

Supervisados

No supervisado

3. Según la búsqueda:

Completa $O(2^N)$
Heurística $O(N^2)$
Aleatoria ¿?

4. Según la salida del algoritmo:

Ranking

Subconjunto de atributos

Selección de Características

Algoritmos Subconjunto de Atributos

Devuelven un subconjunto de atributos optimizado según algún criterio de evaluación.

Entrada: x atributos - U criterio evaluación

Subconjunto = $\{\}$

Repetir

$S_k = \text{generarSubconjunto}(x)$

si existeMejora(S, S_k, U)

Subconjunto = S_k

Hasta CriterioParada()

Salida: Lista, atts más relevantes al principio

Selección de Características

Algoritmos de Ranking

Devuelven una lista de atributos ordenados según algún criterio de evaluación.

Entrada: x atributos - U criterio evaluación

Lista = {}

Para cada Atributo x_i , $i \in \{1, \dots, N\}$

$v_i = \text{calcular}(x_i, U)$

situar x_i dentro de Lista conforme v_i

Salida: Lista, atts más relevantes al principio

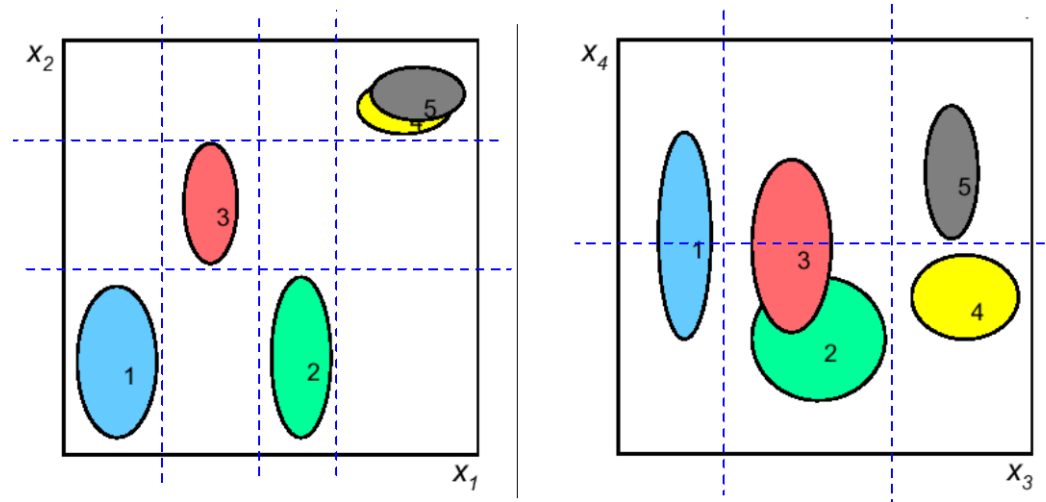
Selección de Características

Validación algoritmos ranking

Atributos	A1	A2	A3	A4	A5	A6	A7	A8	A9
Ranking	A5	A7	A4	A3	A1	A8	A6	A2	A9
	A5	A7	A4	A3	A1	A8	(6 atributos)		

Selección de Características

Algunos algoritmos



$X1: [1,2,3,\{4,5\}]$
 $X2: [\{1,2\},3,\{4,5\}]$
 $X3: [1,\{2,3\},\{4,5\}]$
 $X4: [\{1,2,3\},4,5]$

Es razonable que la función objetivo ofrezca un resultado como

$$f(X1) > f(X2) \approx f(X3) > f(X4)$$

→ Elegir $\{X1, X2\}$ o $\{X1, X3\}$

Parece razonable que se elija $\{X1, X4\}$

Selección de Características

Algunos algoritmos

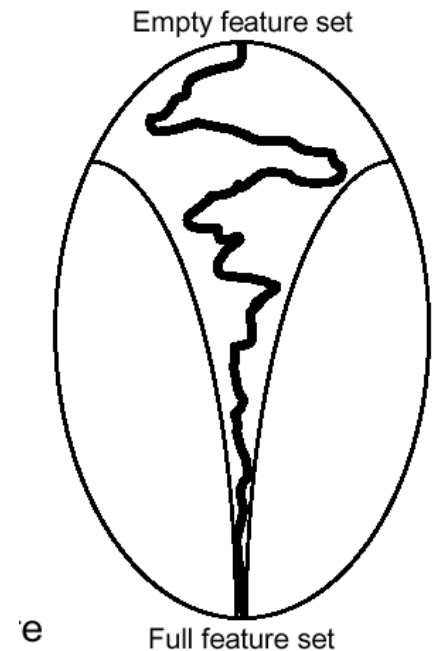
Selección hacia delante

La selección *forward* comienza con el conjunto vacío y de forma secuencial añade al subconjunto actual S el atributo X_i que maximiza $f(S, X_i)$

1. Comenzar con $S = \emptyset$
2. Seleccionar la variable

$$X^+ = \arg \max_{X \in U - S} f(S \cup X)$$

3. $S = S \cup \{X^+\}$
4. Ir al paso 2



Selección de Características

Algunos algoritmos

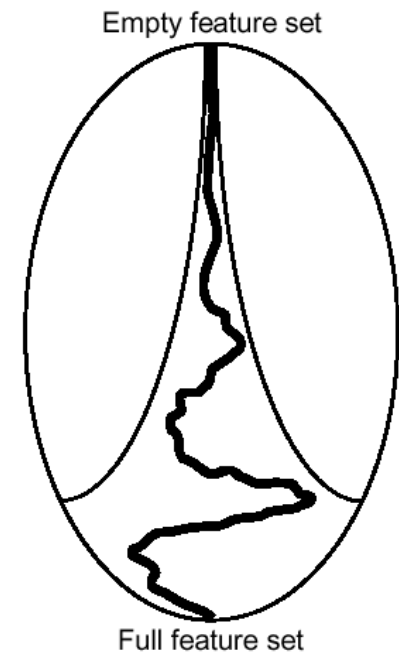
Selección hacia atrás

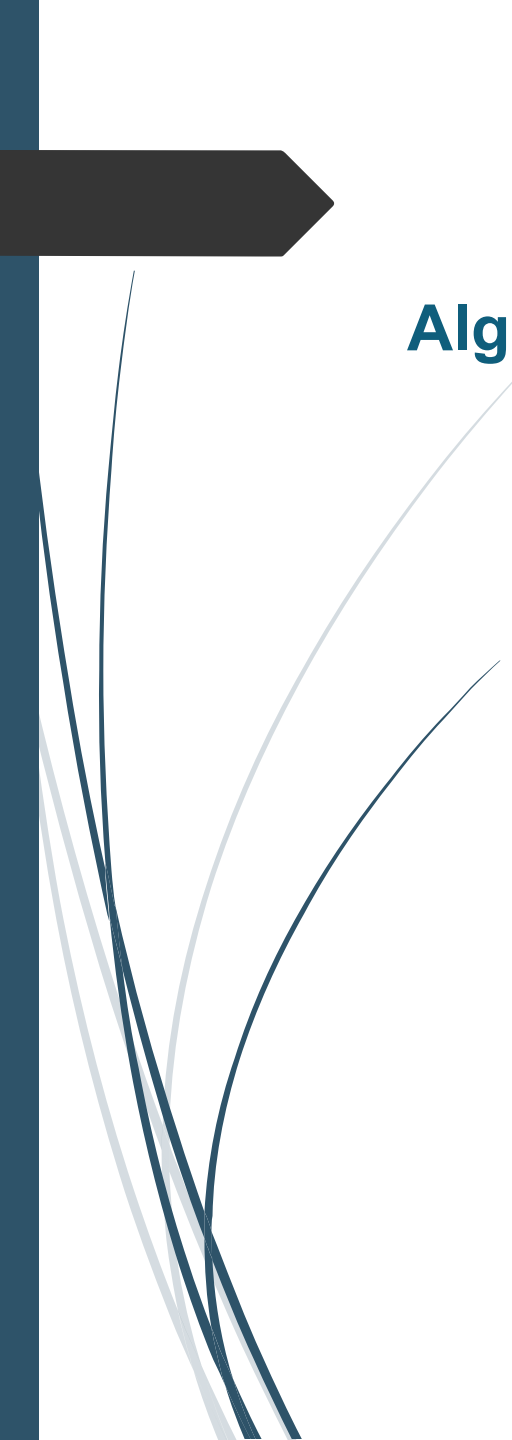
- La selección *backward* comienza con el conjunto completo U y de forma secuencial elimina del subconjunto actual S el atributo X que decrementa menos $f(S-X)$

1. Comenzar con $S=U$
2. Seleccionar la variable X^-

$$X^- = \arg \max_{X \in S} f(S - X)$$

3. $S = S - \{X^-\}$
4. Ir al paso 2





Selección de Características

Algunos algoritmos

- Selección hacia delante:
 - Funciona mejor cuando el subconjunto óptimo tiene pocas variables
 - Es incapaz de eliminar variables
- Selección hacia atrás:
 - Funciona mejor cuando el subconjunto óptimo tiene muchas variables
 - El principal inconveniente es el de reevaluar la utilidad de algunos atributos previamente descartados
- Especialmente con el enfoque envolvente, ¿cuál sería computacionalmente más eficiente?

Selección de Características

Algunos algoritmos

Selección l-más r-menos

- Es una generalización de forward y backward

1. Si $l > r$ entonces $S = \emptyset$
si no, $S = U$ e ir al paso 3
2. Repetir l veces

$$X^+ = \arg \max_{X \in U - S} f(S \cup X)$$

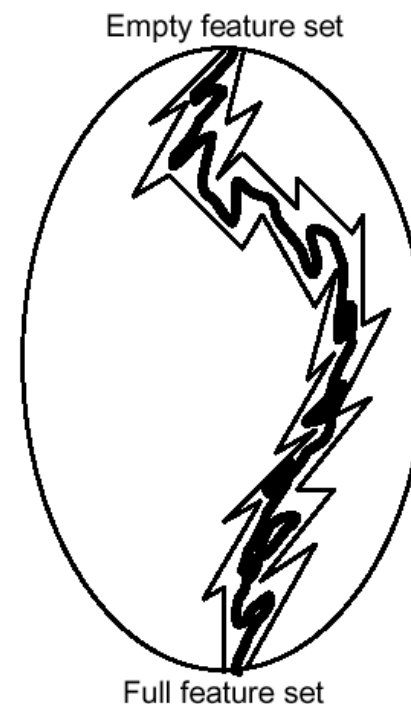
$$S = S \cup \{X^+\}$$

3. Repetir r veces

$$X^- = \arg \max_{X \in S} f(S - X)$$

$$S = S - \{X^-\}$$

4. Ir al paso 2

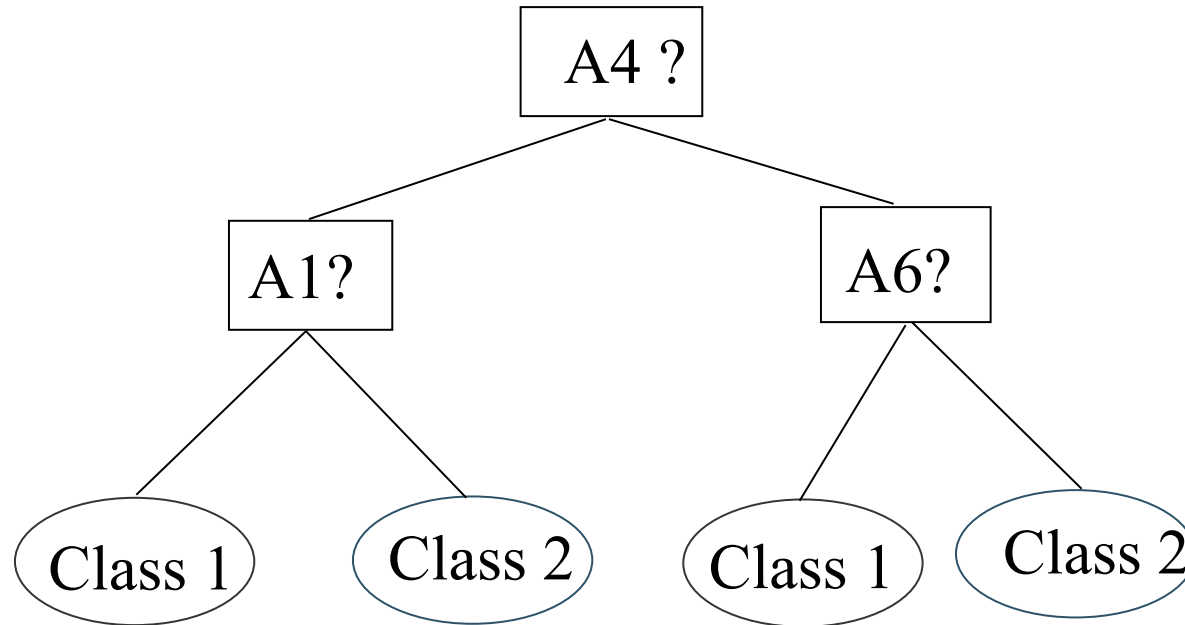


Selección de Características

Algunos algoritmos

Selección de características con árboles de decisión

Conjunto inicial de atributos: {A1, A2, A3, A4, A5, A6}



Características seleccionadas: {A1, A4, A6}

Selección de Características

Algunos algoritmos

Algunos algoritmos relevantes:

- **Algoritmos secuenciales.** Añaden o eliminan variables al subconjunto candidato de forma secuencial. Suelen quedarse en óptimos locales
 - Selección hacia delante, selección hacia atrás, selección más-menos-r, búsqueda bidireccional, selección secuencial flotante
- **Algoritmos exponenciales.** El número de subconjuntos evaluados aumenta exponencialmente con la dimensionalidad del espacio de búsqueda
 - Branch and bound, beam search
- **Algoritmos estocásticos.** Utilizan aleatoriedad para escapar de óptimos locales
 - Ascensión de colinas con reinicios, enfriamiento estocástico, algoritmos genéticos, enfriamiento simulado

Selección de Características

Algunos algoritmos

Table 7.3 All possible combinations for FS algorithms

Search direction	Evaluation measure	Search strategy		
		Exhaustive	Heuristic	Nondeterministic
Forward	Probability	C1	C7	–
	Consistency	C2	C8	–
	Accuracy	C3	C9	–
Backward	Probability	C4	C10	–
	Consistency	C5	C11	–
	Accuracy	C6	C12	–
Random	Probability	–	C13	C16
	Consistency	–	C14	C17
	Accuracy	–	C15	C18



Selección de Características

Algunos algoritmos

- **Focus algorithm.** Consistency measure for forward search,
- Mutual Information based Features Selection (MIFS).
- Las Vegas Filter (LVF)
- Las Vegas Wrapper (LVW)
- Relief Algorithm
- mRMR: Minimum Redundancy Maximum Relevance

Selección de Características

Algunos algoritmos

Algorithm 6 LVF algorithm.

function LVF(D - a data set with M features, U - the inconsistency rate, $maxTries$ - stopping criterion, γ - an allowed inconsistency rate)

initialize: list $L = \{\}$

 ▷ L stores equally good sets

$C_{best} = M$

for $maxTries$ iterations **do**

$S = \text{RANDOMSET}(\text{seed})$

$C = \#(S)$

 ▷ $\#$ - the cardinality of S

if $C < C_{best}$ and $\text{CALU}(S, D) < \gamma$ **then**

$S_{best} = S$

$C_{best} = C$

$L = \{S\}$

 ▷ L is reinitialized

else if $C = C_{best}$ and $\text{CALU}(S, D) < \gamma$ **then**

$L = \text{APPEND}(S, L)$

end if

end for

return L

 ▷ all equivalently good subsets found by LVF

end function

Selección de Características

Algunos algoritmos

Algorithm 8 MIFS algorithm.

```
function MIFS( $F$  - all features in data,  $S$  - set of selected features,  $k$  - desired size of  $S$ ,  
 $\beta$  - regulator parameter)  
  initialize:  $S = \{\}$   
  for each feature  $f_i$  in  $F$  do  
    Compute  $I(C, f_i)$   
  end for  
  Find  $f_{max}$  that maximizes  $I(C, f)$   
   $F = F - \{f_{max}\}$   
   $S = S \cup f_{max}$   
  repeat  
    for all couples of features  $(f_i \in F, s_j \in S)$  do  
      Compute  $I(f_i, s_j)$   
    end for  
    Find  $f_{max}$  that maximizes  $I(C, f) - \beta \sum_{s \in S} I(f_i, s_j)$   
     $F = F - \{f_{max}\}$   
     $S = S \cup f_{max}$   
  until  $|S| = k$   
  return  $S$   
end function
```

Selección de Características

Algunos algoritmos

Algunos algoritmos relevantes:

- Algoritmos estocásticos. Algoritmos meteheurísticos
 - Enfriamiento simulado
 - Algoritmos evolutivos

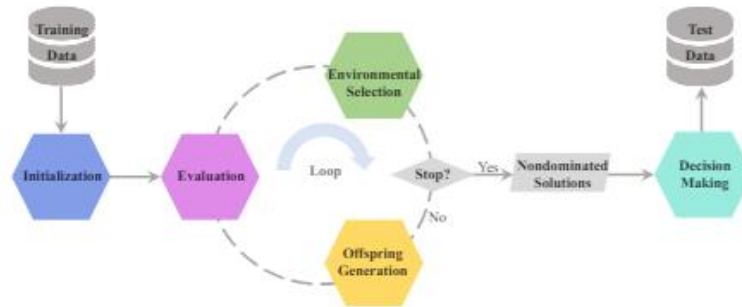


Fig. 2. Framework of MOFS.

The mapping of the Pareto set in the objective space is called the Pareto front:

$$PF = \{\mathbf{F}(\mathbf{x}) | \mathbf{x} \in PS\}.$$

Journals & Magazines > IEEE Transactions on Evolutio... > Early Access

A Survey on Evolutionary Multiobjective Feature Selection in Classification: Approaches, Applications, and Challenges

Publisher: IEEE

[Cite This](#)

[PDF](#)

Ruwang Jiao ; Bach Hoai Nguyen ; Bing Xue ; Mengjie Zhang [All Authors](#)

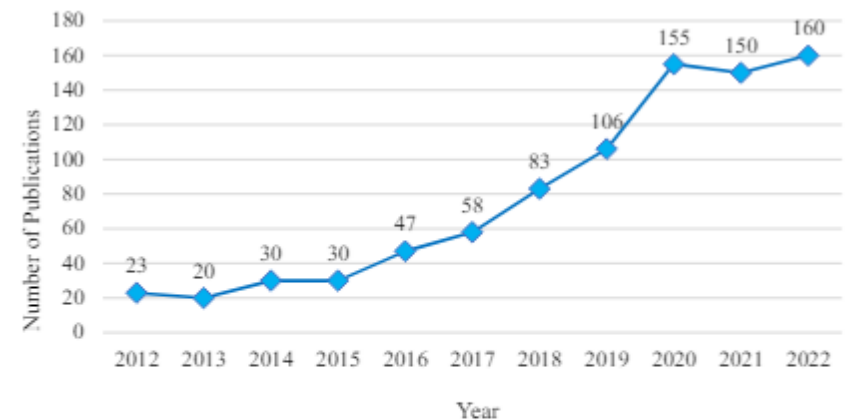


Fig. 1. Number of publications on MOFS in classification from 2012 to 2022. The data is collected from the Web of Science on January 6, 2023, with the topics of “multi-objective feature selection” OR “multiobjective feature selection” AND “classification”. The total number of publications from 2012 to 2022 is 862.

Selección de Características

Fig. 3. Taxonomy of MOFS according to different criteria.

Binary encoding:

F_1	F_2	F_3	F_4	F_5	F_6
1	0	1	1	0	0

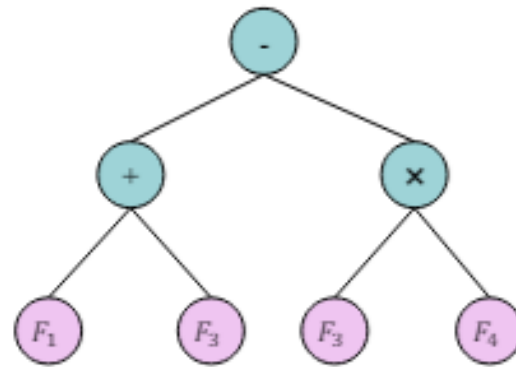
Selected feature = $\{F_1, F_3, F_4\}$

Continuous encoding:

F_1	F_2	F_3	F_4	F_5	F_6
0.7	0.2	0.9	0.6	0.1	0.3

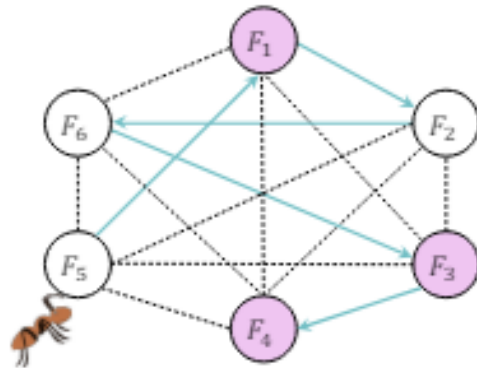
> 0.5
Selected feature = $\{F_1, F_3, F_4\}$

(a) Vector representation



Selected feature = $\{F_1, F_3, F_4\}$

(b) Tree representation



Selected feature = $\{F_1, F_3, F_4\}$

(c) Graph representation

Matrix			Feature score	
Feature number	1	2		
1	0.9190	0.4863	-0.9077	1.38
2	0.5844	0.5155	-0.4462	0.90
3	-0.9286	0.3110	0.6469	1.17
4	0.6983	-0.6576	0.3897	1.04
5	0.3115	-0.2155	-0.8057	0.89
6	0.8680	0.4121	-0.3658	1.03
	1	2	3	
Class label				

The order in which features are selected:
 $\{F_1, F_3, F_4, F_6, F_2, F_5\}$

(d) Matrix representation

Selección de características

Extracción de características

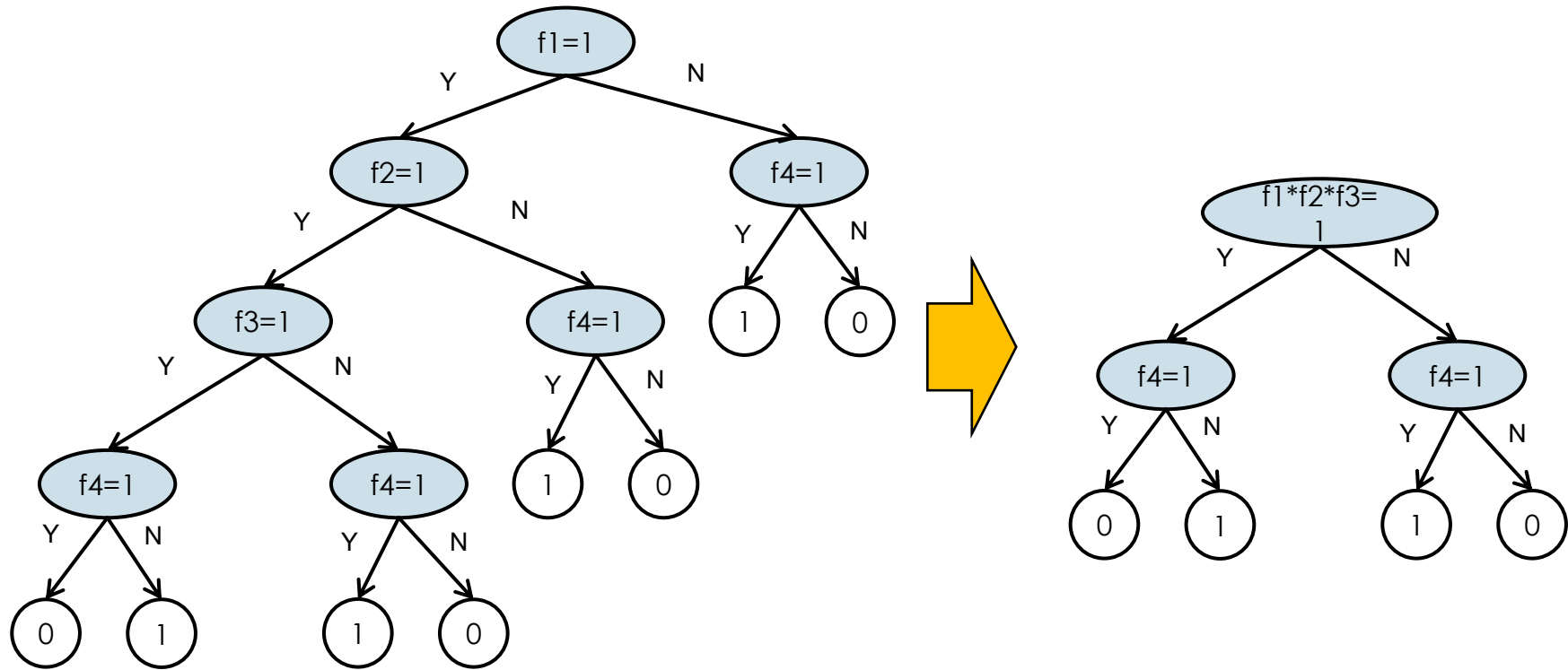


Fig. 7.4 The effect of using the product of features in decision tree modeling



Calidad de Datos

Calidad de datos

Introducción al preprocesamiento de datos

Integración, limpieza y transformación

Datos imperfectos

Reducción de datos

Discretización

Selección de Características

Selección de Instancias

Comentarios finales

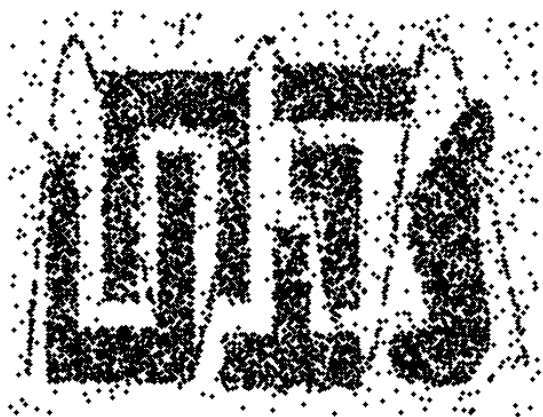
Suplemento: Reducción de Dimensionalidad

Selección de Instancias

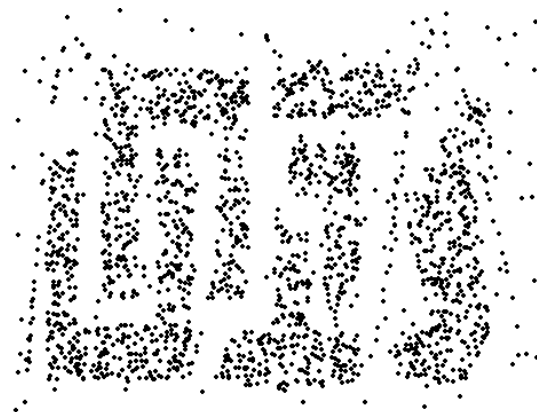
- ✿ La SI pretende elegir los ejemplos que sean relevantes para una aplicación y lograr el máximo rendimiento. El resultado de la SC sería:
 - ❖ Menos datos → los algoritmos pueden aprender más rápidamente
 - ❖ Mayor exactitud → el clasificador generaliza mejor
 - ❖ Resultados más simples → más fácil de entender
- ✿ SI y Transformación (compactación/agrupamiento)

Selección de Instancias

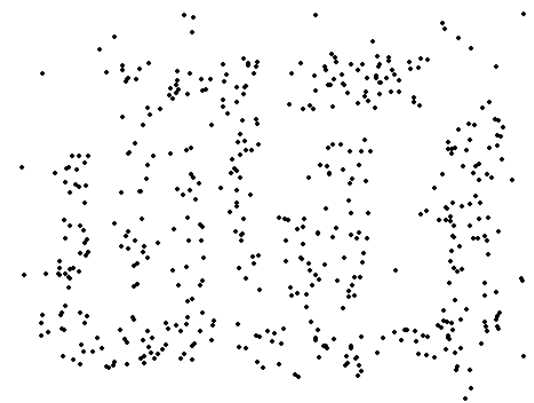
Ejemplos de diferentes tamaños



8000 puntos

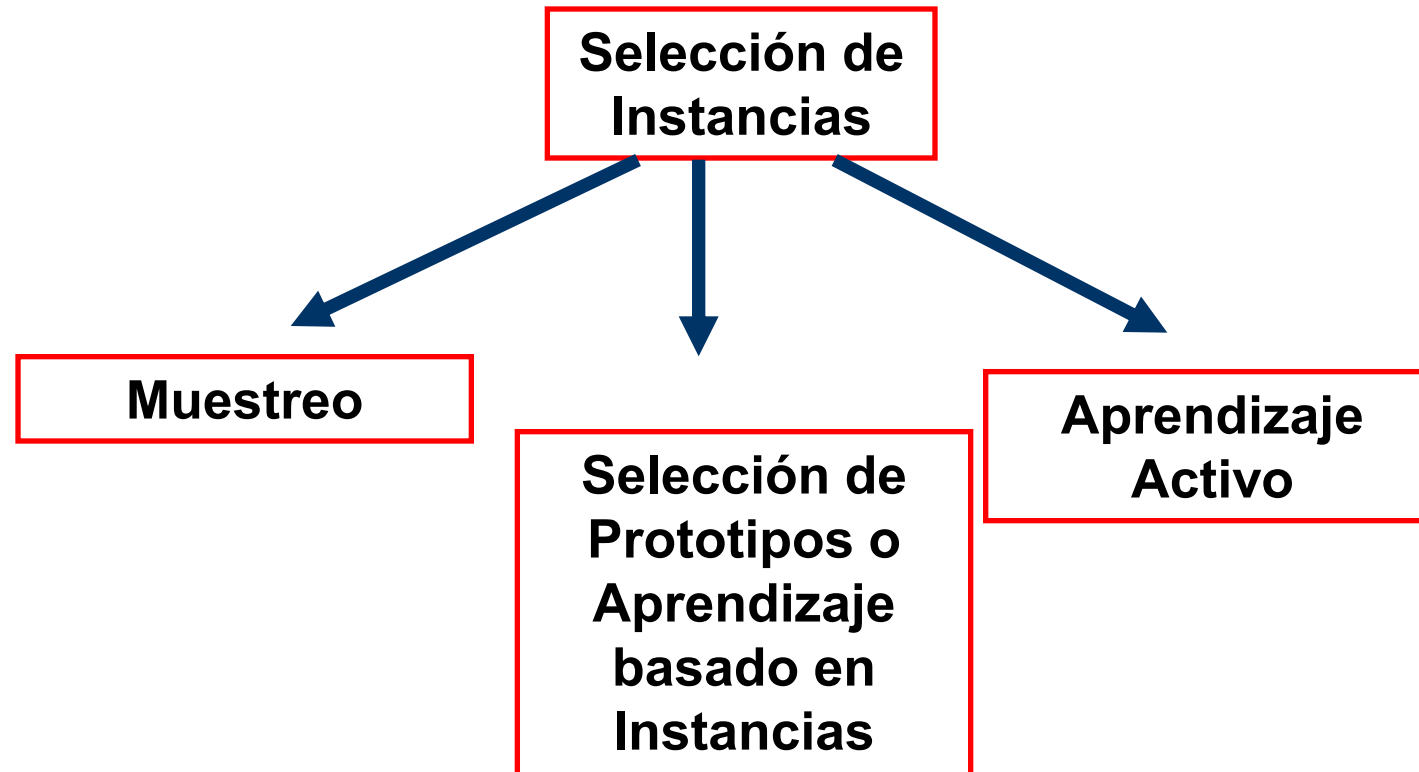


2000 puntos

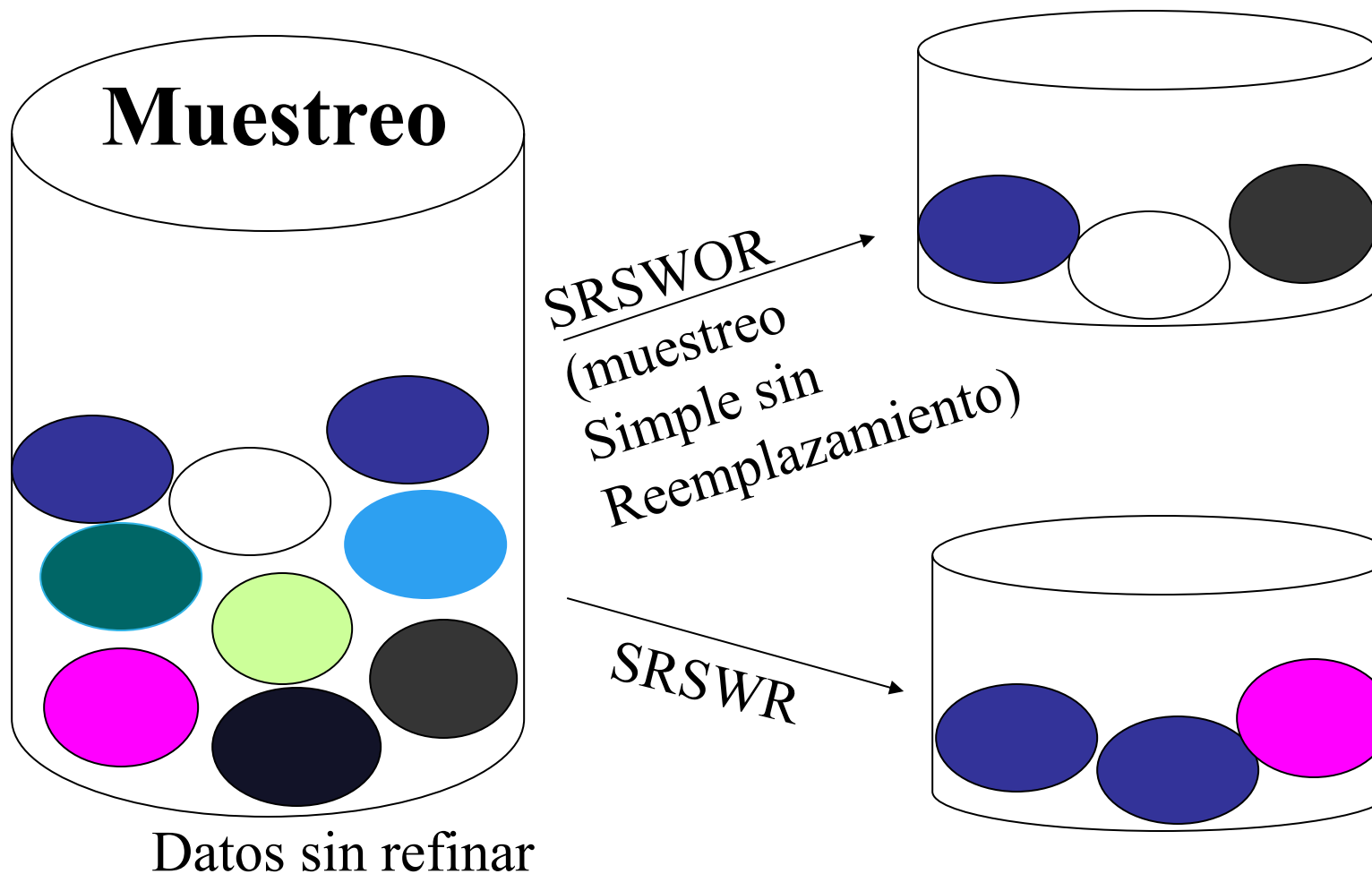


500 puntos

Selección de Instancias

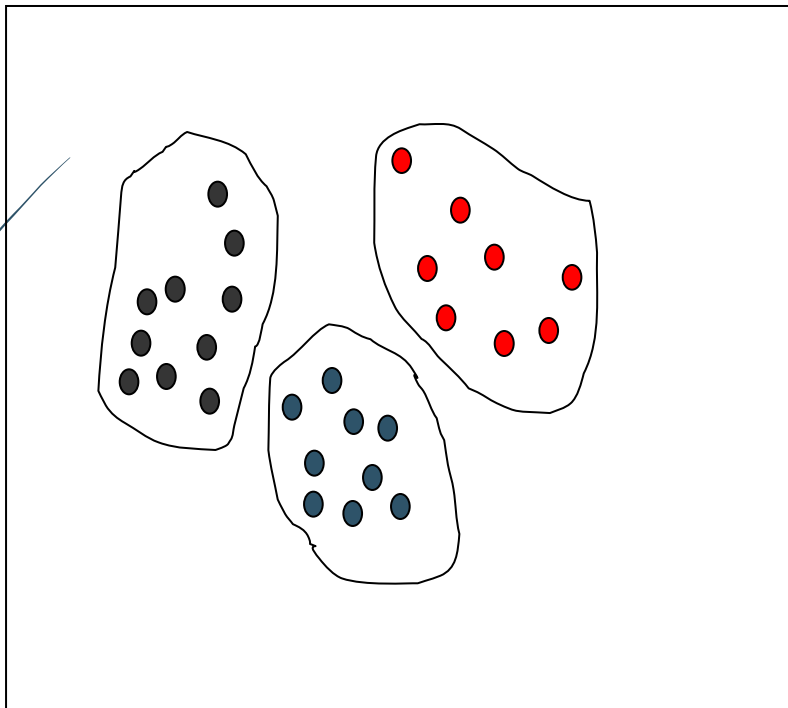


Selección de Instancias

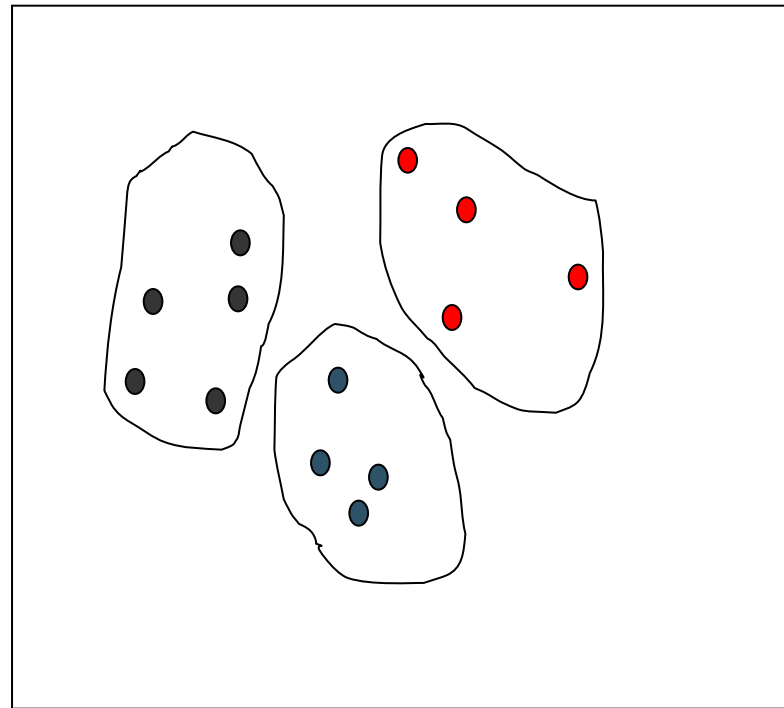


Selección de Instancias

Datos sin refinar



Reducción simple



Selección de Instancias

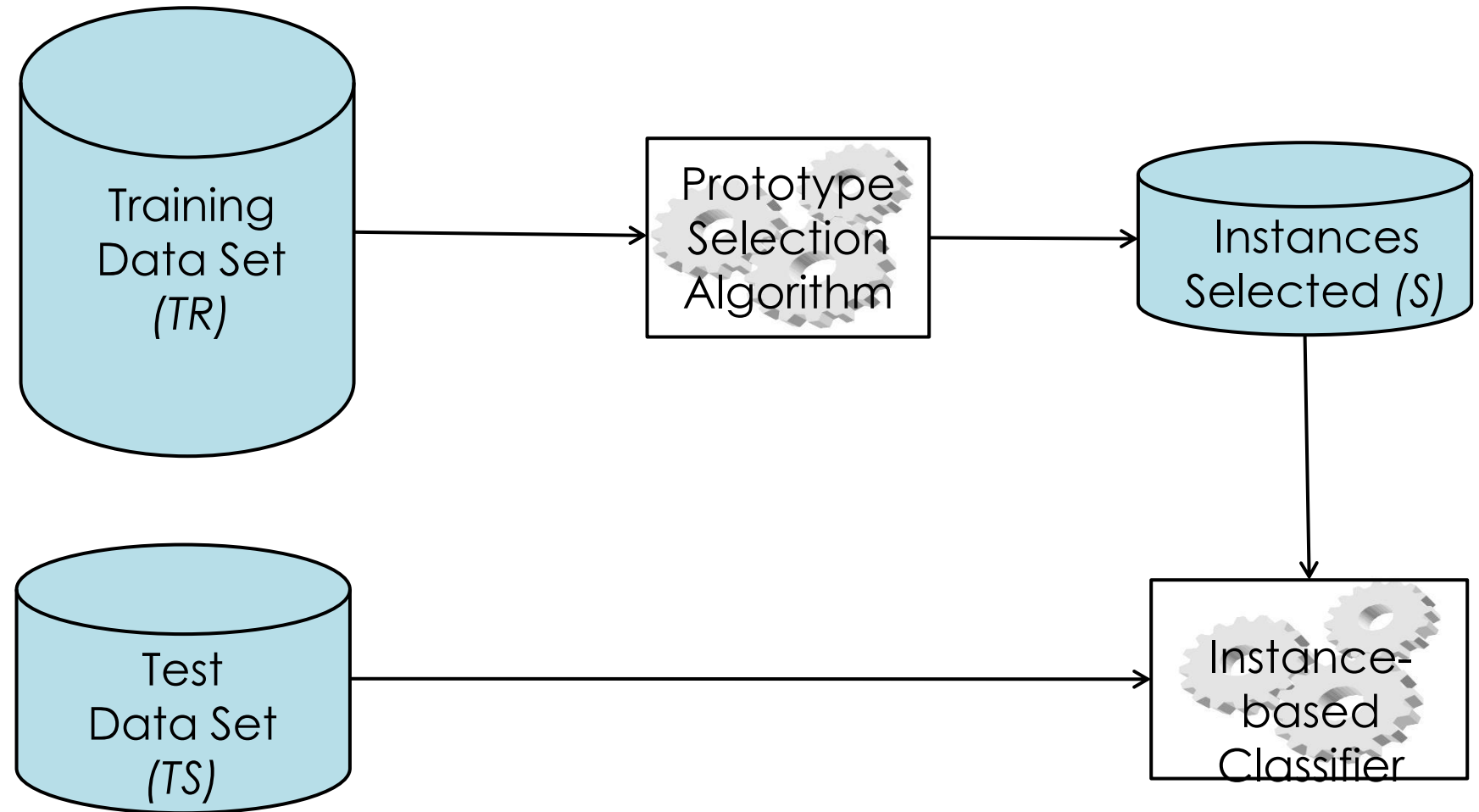
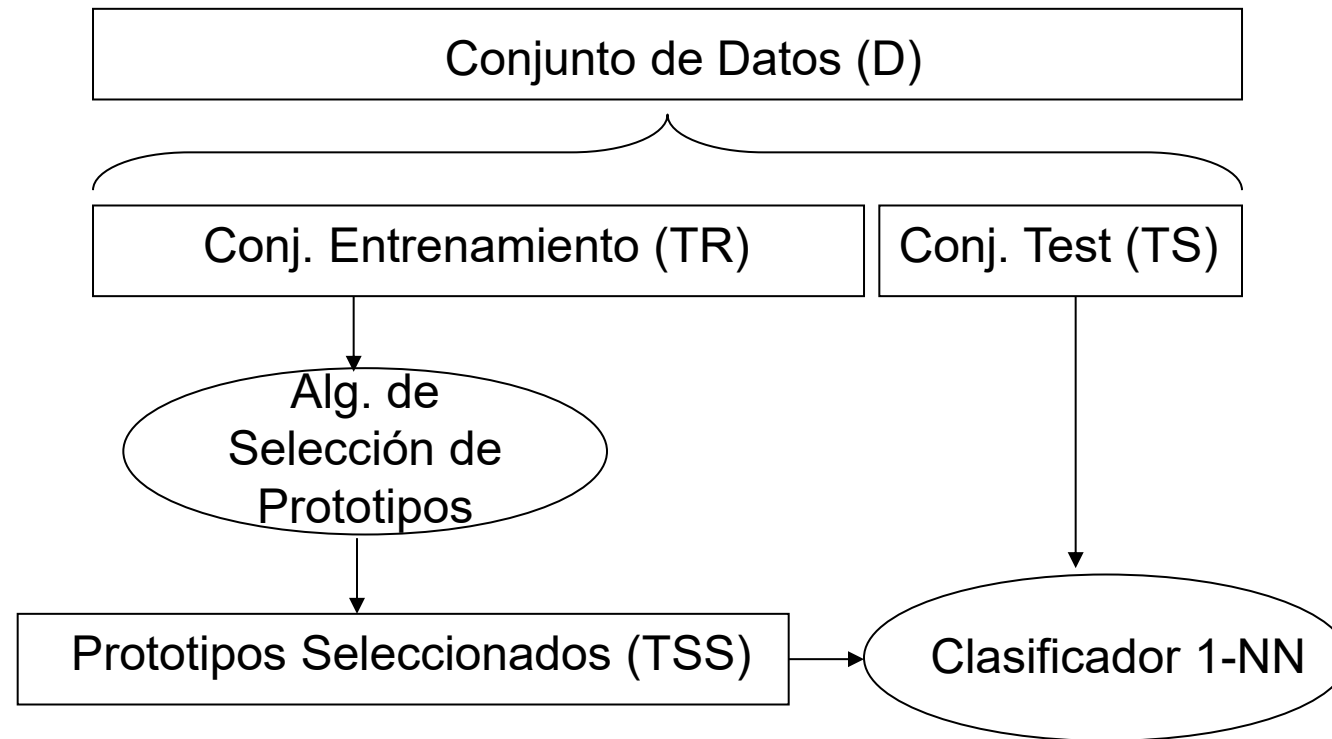
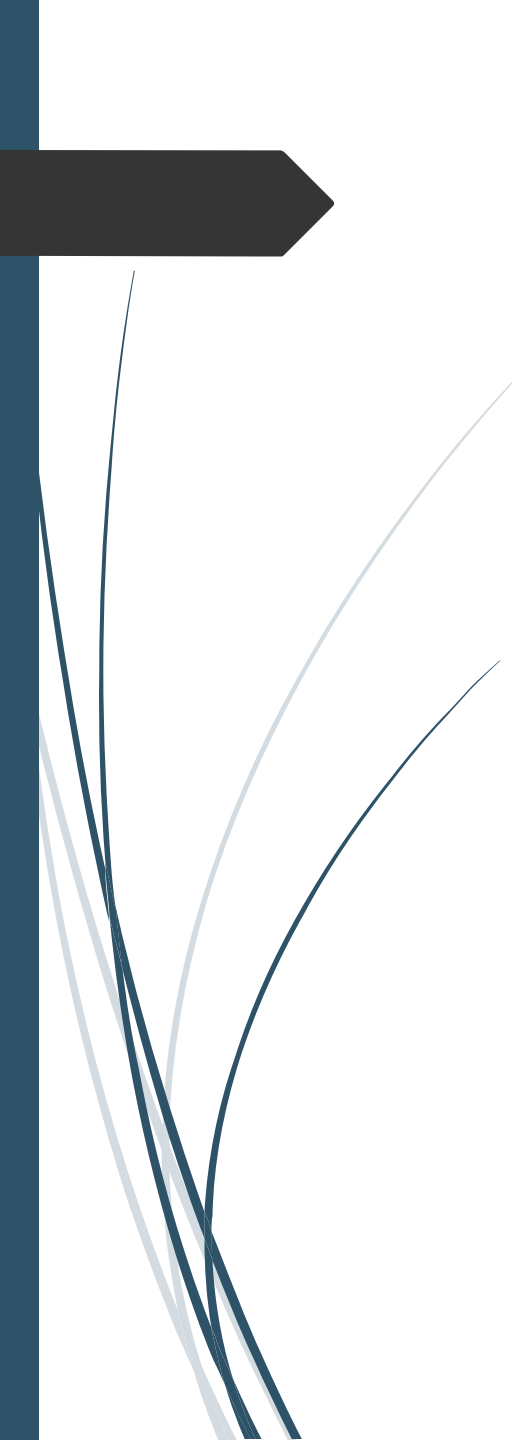


Fig. 8.1 Prototype Selection process

Selección de Instancias

Selección de Prototipos para Clasificación con 1-NN





Selección de Instancias

Formas de evaluar un algoritmo de Selección de instancias en k-NN:

- Reducción del espacio de almacenamiento
- Tolerancia al ruido
- Precisión en la generalización del aprendizaje
- Requerimientos de cómputo



Selección de Instancias

Selección de Prototipos

Propiedades:

- Dirección de la búsqueda: Incremental, decremental, por lotes, mezclada y fijada.
 - Tipo de selección: Condensación, Edición, Híbrido.
 - Tipo de evaluación: Filtrada o envolvente.
- 

Selección de Instancias

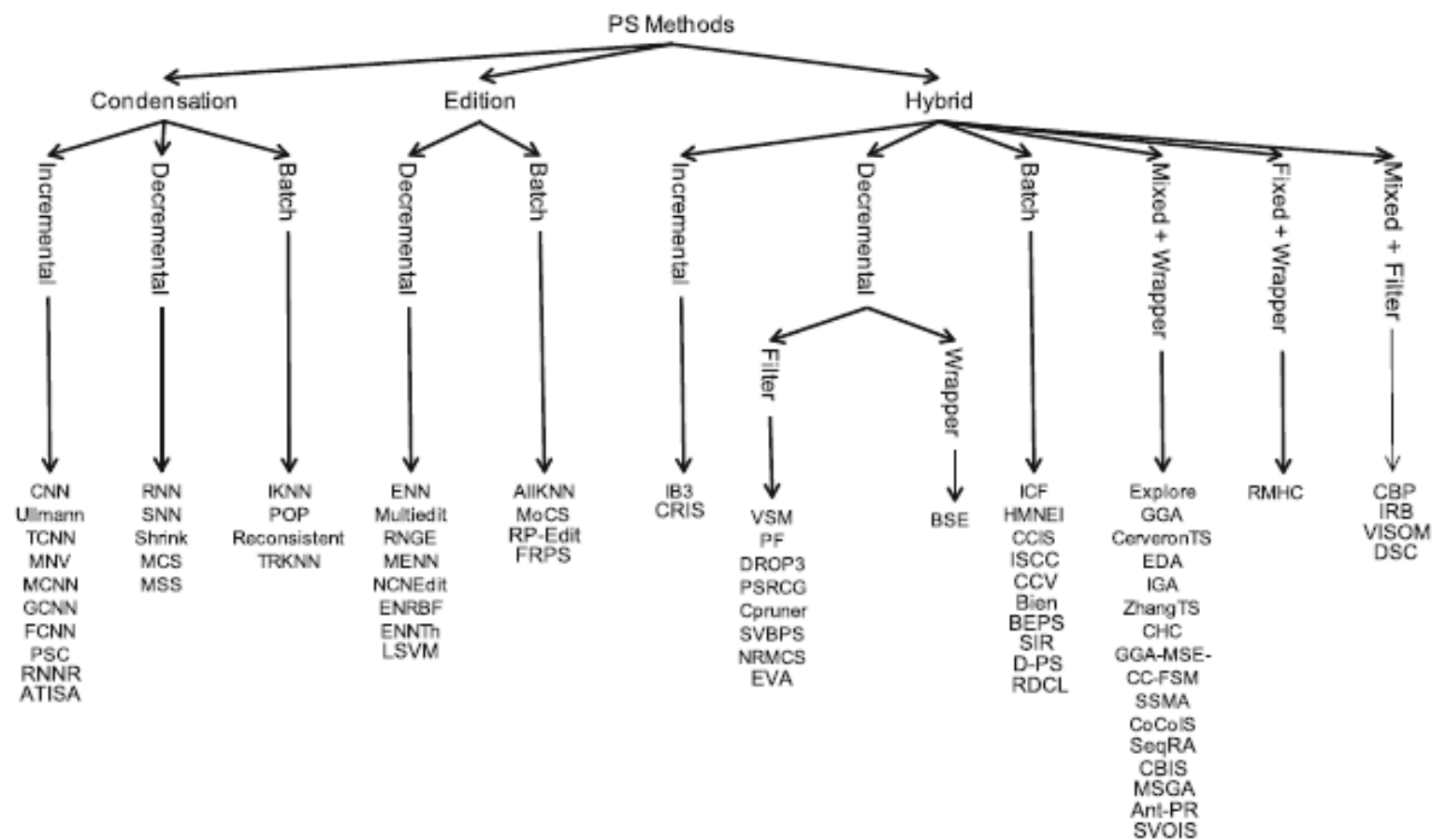


Fig. 8.3 PS taxonomy

Selección de Instancias

- Algoritmo clásico de Condensación: Condensed Nearest Neighbor (CNN)
 - Incremental
 - Inserta solo las instancias mal clasificadas a partir de una selección aleatoria de una instancia de cada clase.
 - Dependiente del orden de presentación
 - Solo retiene puntos pertenecientes al borde

Algorithm 10 CNN algorithm.

```
function CNN( $T$  - training data)
  initialize:  $S = \emptyset$ 
  repeat
    for all  $x \in T$  (in random order) do
      Find  $x' \in S$  s.t.  $\|x - x'\| = \min_{x' \in S} \|x - x'\|$ 
      if  $\text{class}(x) \neq \text{class}(x')$  then
         $S = S \cup \{x\}$ 
      end if
    end for
  until  $S$  does not change
  return  $S$ 
end function
```

Selección de Instancias

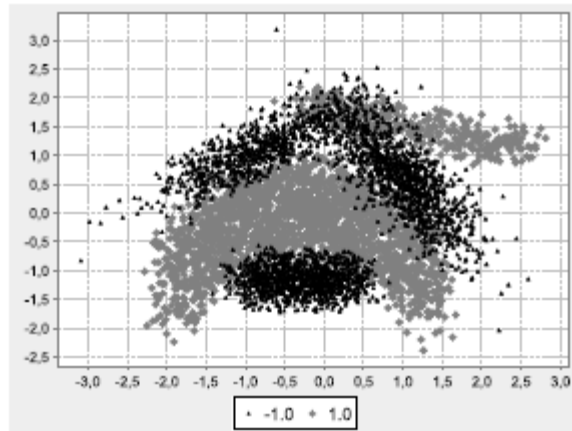
- Algoritmo clásico de Edición: Edited Nearest Neighbor (ENN)
 - Por lotes
 - Se eliminan aquellas instancias que se clasifican incorrectamente usando sus k vecinos más cercanos ($K = 3, 5$ ó 9).
 - “Suaviza” las fronteras, pero retiene el resto de puntos (muchos redundantes)

Algorithm 11 ENN algorithm.

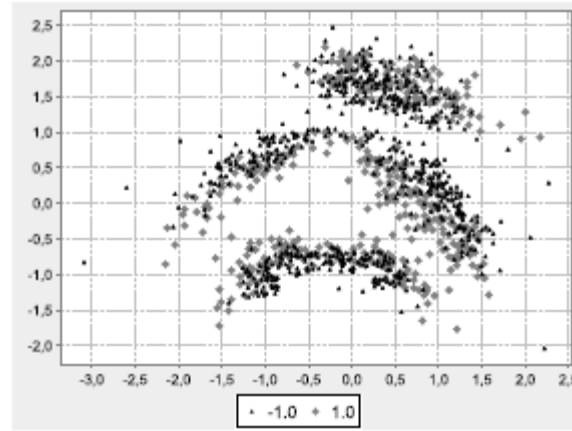
```
function ENN( $T$  - training data,  $k$  - number of nearest neighbor)
  initialize:  $S = T$ 
  for all  $x \in S$  do
     $X' = \emptyset$ 
    for  $i = 1$  to  $k$  do
      Find  $x'_i \in T$  s.t.  $x \neq x'_i$  and  $\|x - x'_i\| = \min_{w \in (T \setminus X')} \|x - x^j\|$ 
       $X' = X' \cup \{x'_i\}$ 
    end for
    if  $\text{class}(x) \neq \text{majorityClass}(X')$  then
       $S = S \setminus \{x\}$ 
    end if
  end for
  return  $S$ 
end function
```

Selección de Instancias

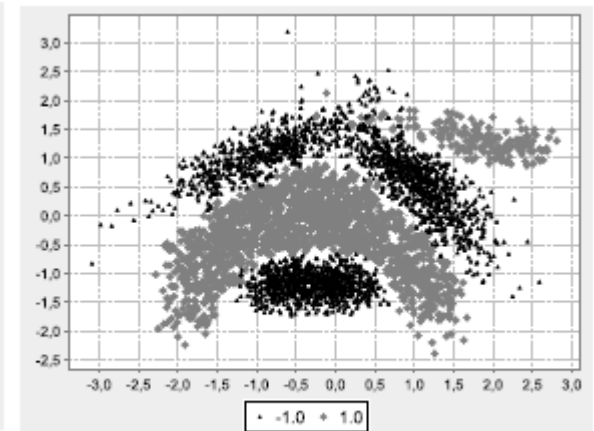
Ejemplos gráficos:



(a) Banana Original
(0.8751, 0.7476)



(b) CNN (0.7729, 0.8664, 0.7304)

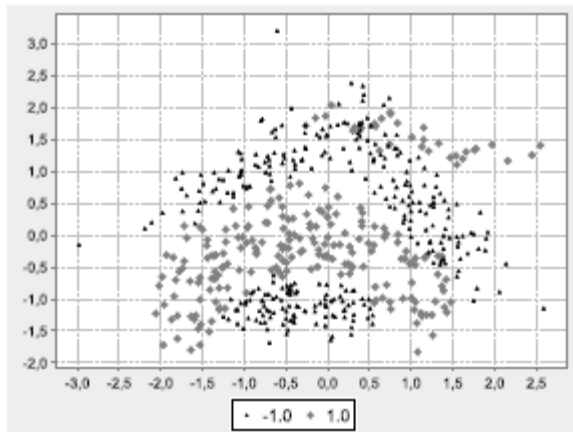


(h) AllKNN (0.1758, 0.8934, 0.7831)

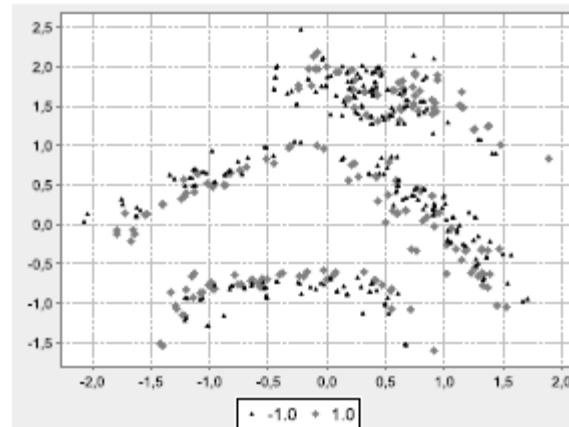
Conjunto banana con 5.300 instancias y dos clases. Conjunto obtenido por CNN y AllKNN (aplicación iterativa de ENN con $k=3, 5$ y 7).

Selección de Instancias

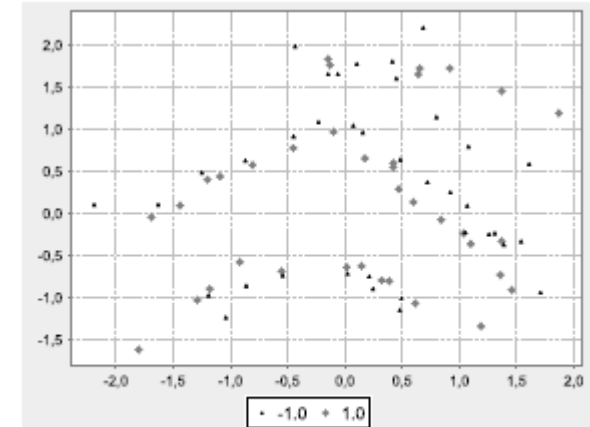
Ejemplos gráficos:



(k) RMHC (0.9000, 0.8972, 0.7915)



(e) DROP3 (0.9151, 0.8696, 0.7356)



(l) SSMA (0.9879, 0.8964, 0.7900)

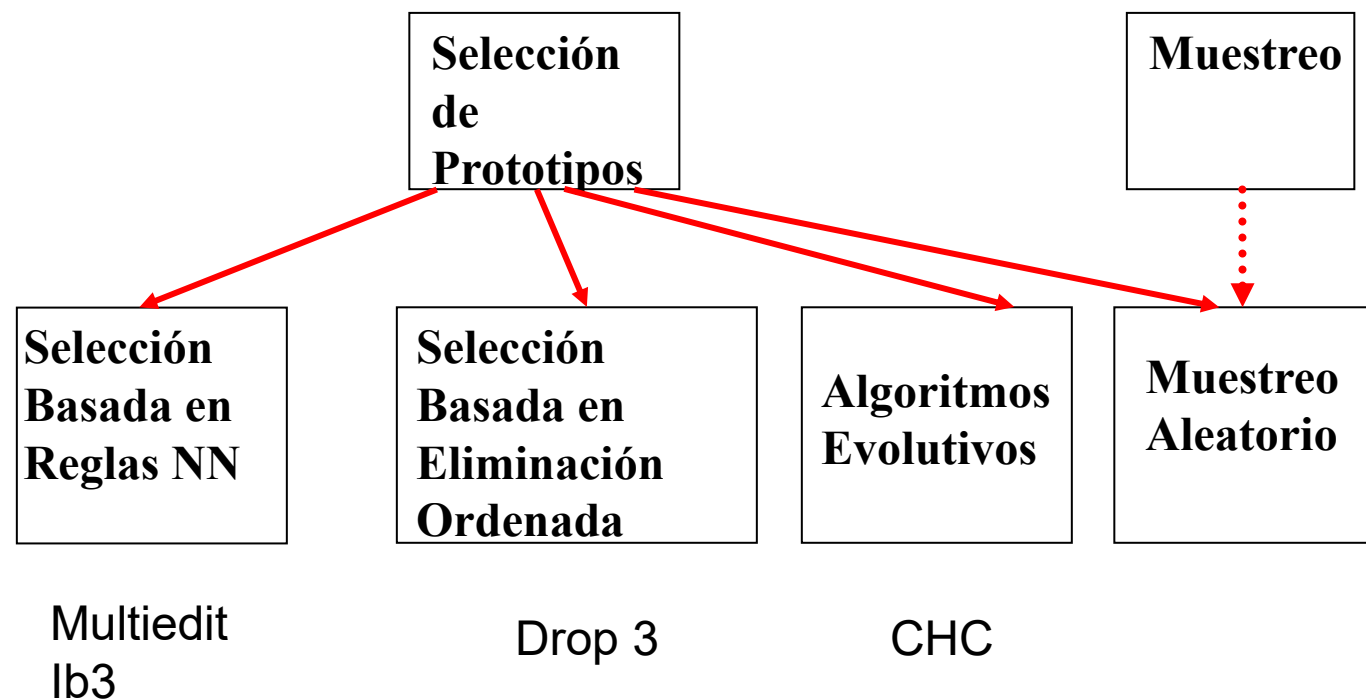
RMHC es una técnica de muestreo adaptativo basa en búsquedas locales con un tamaño final fijo.

DROP3 es la técnica híbrida más conocida y utilizada para NN.

SSMA es una aproximación evolutiva basada en algoritmo meméticos.

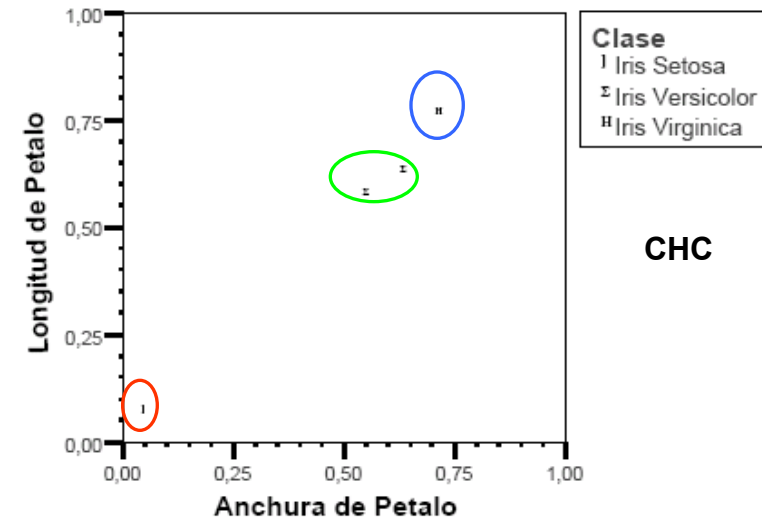
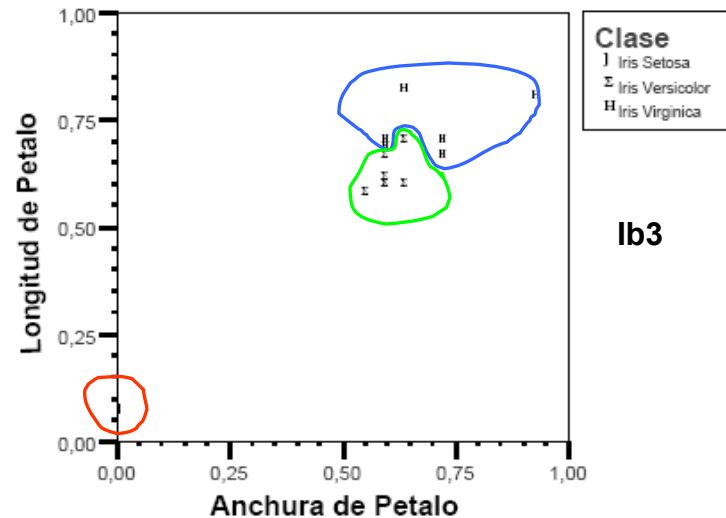
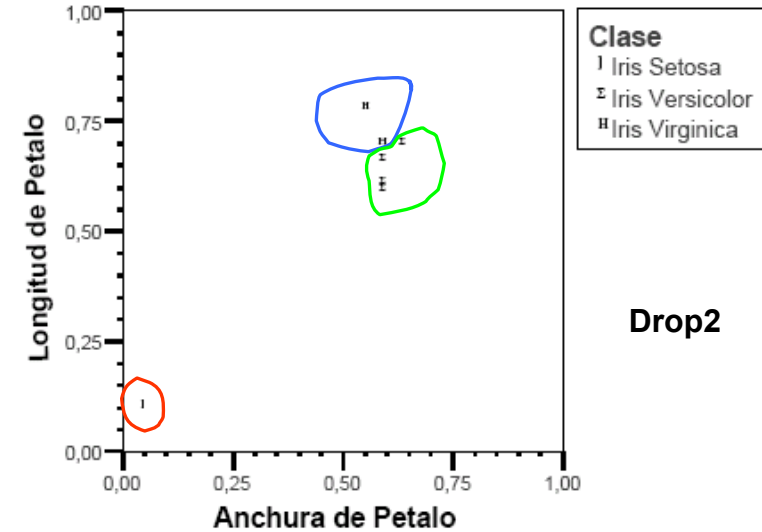
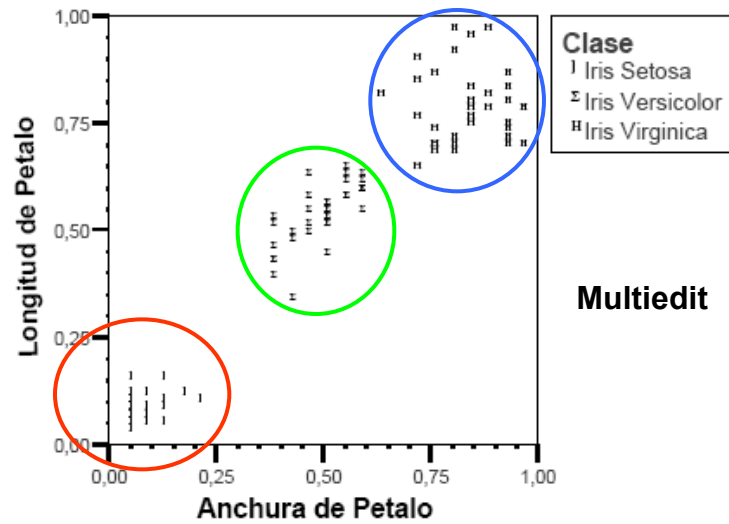
Selección de Instancias

Ejemplos gráficos:



Selección de Instancias

Ejemplos gráficos:



Bibliografía: J.R. Cano, F. Herrera, M. Lozano. Using Evolutionary Algorithms as Instance Selection for Data Reduction in KDD: An Experimental Study. IEEE Trans. on Evolutionary Computation 7:6 (2003) 561-575.

Selección de Instancias

Selección de Instancias. Eficiencia

El orden de los algoritmos es superior a $O(n^2)$ y suele estar en orden $O(n^3)$.

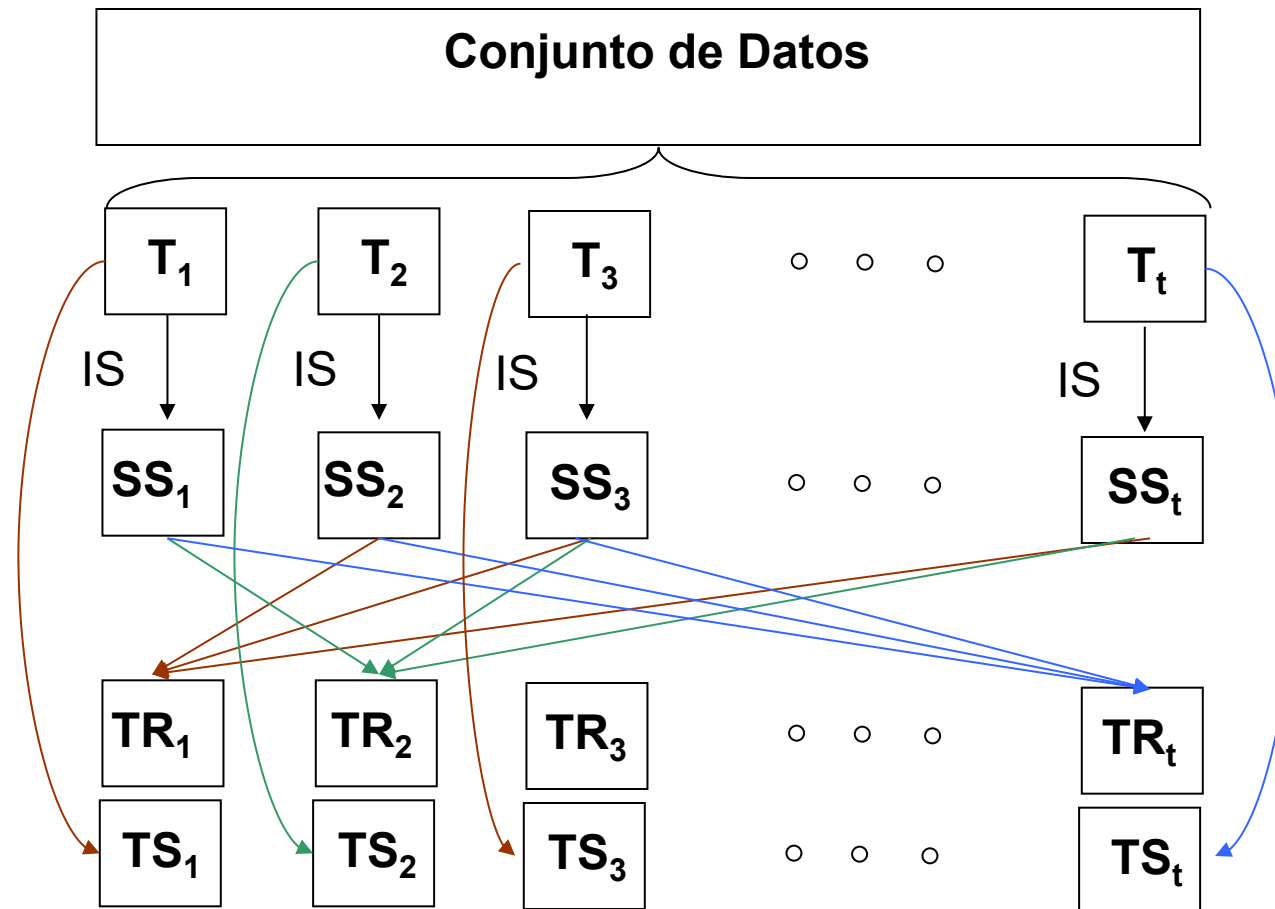
Las principales dificultades que deben afrontar los algoritmos de Selección de Prototipos son: Eficiencia, recursos, generalización, representación.

¿Cómo realizar la selección de instancias con grandes bases de datos?

Combinamos una estrategia de estratificación con los algoritmos de selección de instancias.

Selección de Instancias

Grandes Bases de Datos. Estrategia de Estratificación.



Referencia: J.R. Cano, F. Herrera, M. Lozano. Stratification for Scaling Up Evolutionary Prototype Selection. Pattern Recognition Letters 26:7 (2005) 953-963.



Selección de Instancias

Selección de Instancias. Ejemplo – Kdd Cup'99

Nombre	Número de Instancias	Número de Atributos	Número de Clases
Kdd Cup'99	494022	41	23

Selección de Instancias

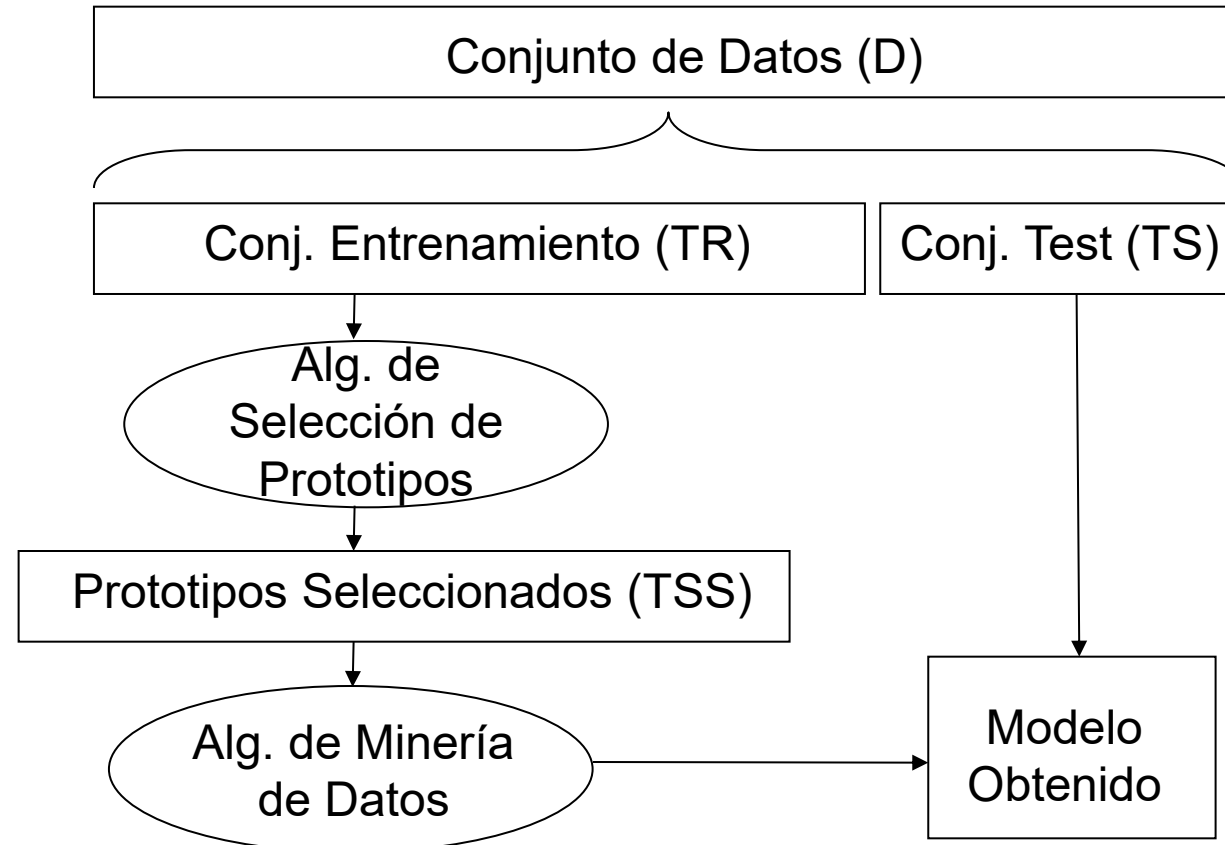
Selección de Instancias. Ejemplo – Kdd Cup'99

	Tiempo	% Red	% Ac. Trn	% Ac Test
1-NN cl	18568		99.91	99.91
Cnn st 100	8	81.61	99.30	99.27
Cnn st 200	3	65.57	99.90	99.15
Cnn st 300	1	63.38	99.89	98.23
Ib2 st 100	0	82.01	97.90	98.19
Ib2 st 200	3	65.66	99.93	98.71
Ib2 st 300	2	60.31	99.89	99.03
Ib3 st 100	2	78.82	93.83	98.82
Ib3 st 200	0	98.27	98.37	98.93
Ib3 st 300	0	97.97	97.92	99.27
CHC st 100	1960	99.68	99.21	99.43
CHC st 200	418	99.48	99.92	99.23
CHC st 300	208	99.28	99.93	99.19

J.R. Cano, [F. Herrera](#), [M. Lozano](#), **Stratification for Scaling Up Evolutionary Prototype Selection**. *Pattern Recognition Letters*, 26, (2005), 953-963.

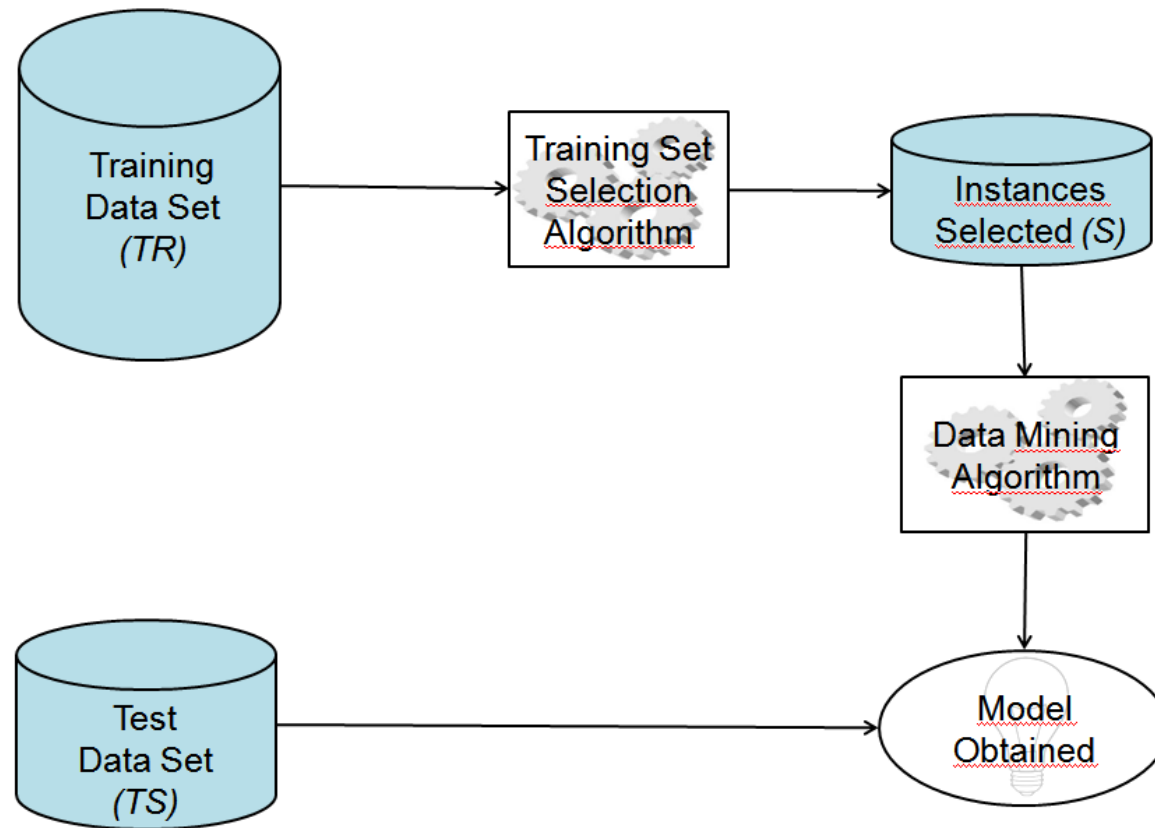
Selección de Instancias

Selección de Prototipos la **Selección de Conjuntos de Entrenamiento**



Selección de Instancias

Selección de Prototipos la **Selección de Conjuntos de Entrenamiento**

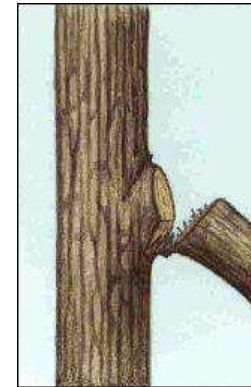


Selección de Instancias



Comprehensibilidad: Árboles de Tamaño reducido

**Se utilizan técnicas de poda
eliminación de nodos**



Las estrategias de selección de instancias permiten construir árboles de decisión para grandes bases de datos reduciendo el tamaño de los árboles.

Aumentan su interpretabilidad.

Selección de Instancias

Kdd Cup'99. Número de estratos: 100

	No. Reglas	% Reducción	<i>C4.5</i>	
			%Ac Trn	%Ac Test
<i>C4.5</i>	252		99.97%	99.94%
<i>Cnn Strat</i>	83	81.61%	98.48%	96.43%
<i>Drop1 Strat</i>	3	99.97%	38.63%	34.97%
<i>Drop2 Strat</i>	82	76.66%	81.40%	76.58%
<i>Drop3 Strat</i>	49	56.74%	77.02%	75.38%
<i>Ib2 Strat</i>	48	82.01%	95.81%	95.05%
<i>Ib3 Strat</i>	74	78.92%	99.13%	96.77%
<i>Icf Strat</i>	68	23.62%	99.98%	99.53%
<i>CHC Strat</i>	9	99.68%	98.97%	97.53%



Calidad de Datos

Calidad de datos

Introducción al preprocesamiento de datos

Integración, limpieza y transformación

Datos imperfectos

Reducción de datos

Discretización

Selección de Características

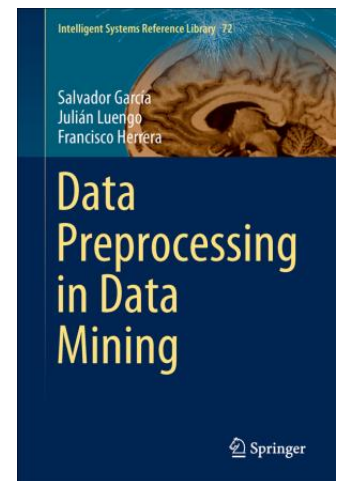
Selección de Instancias

Comentarios finales

Suplemento: Reducción de Dimensionalidad

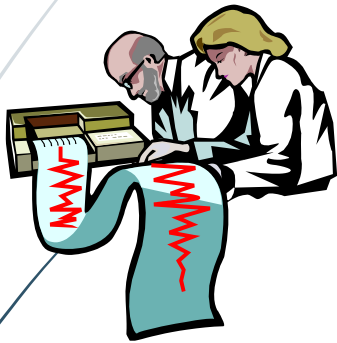
Comentarios finales

El preprocesamiento de datos es una necesidad cuando se trabaja con una aplicación real, con datos obtenidos directamente del problema.



Comentarios finales

Datos sin refinar

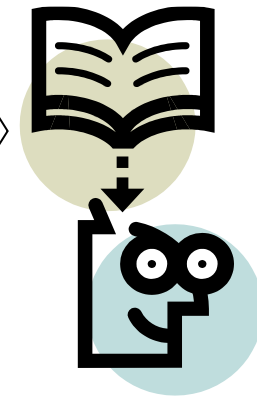


**Preprocesa-
miento
de
Datos**

**Obtención
de
Patrones**

**Interpretación
de
Resultados**

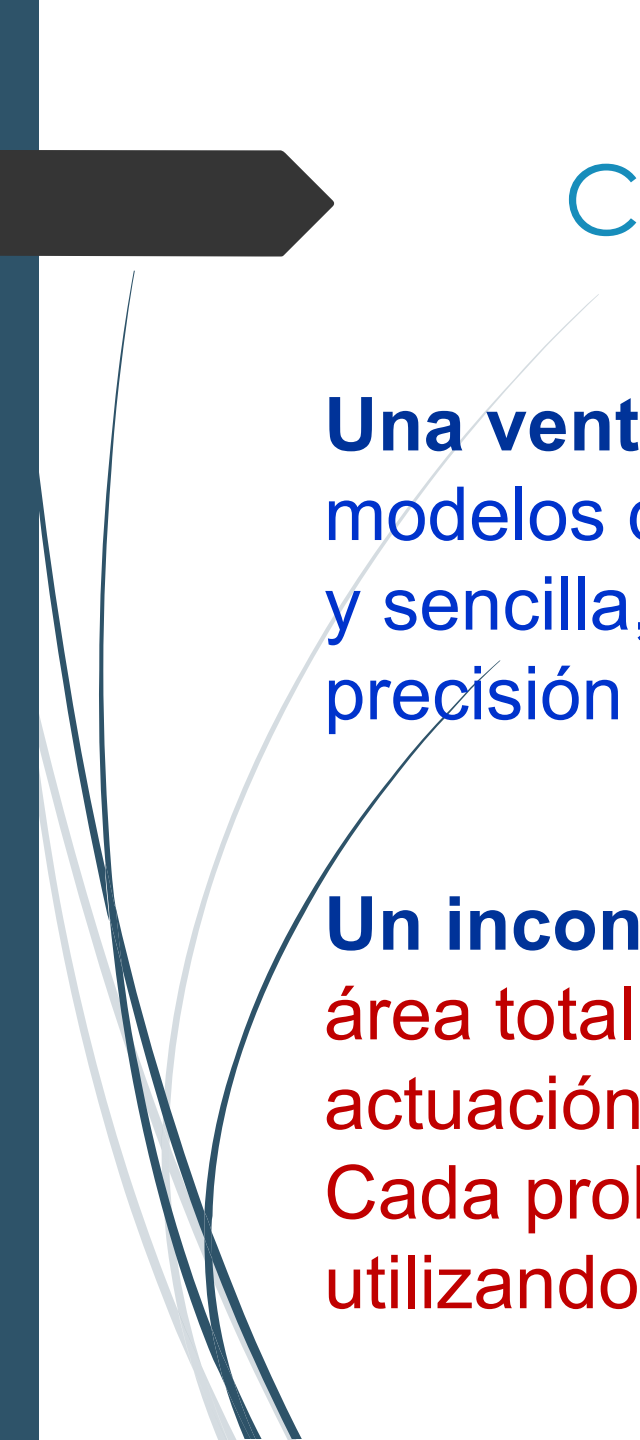
Conocimiento



- Preparación de Datos
- Reducción

- Reglas de asociación
- Clasificación / predicción
- Análisis de cluster

- Visualización
- Validación



Comentarios finales

Una ventaja: El preprocesamiento de datos permite aplicar los modelos de Aprendizaje/Minería de Datos de forma más rápida y sencilla, obteniendo modelos/patrones de más calidad: precisión e/o interpretabilidad.

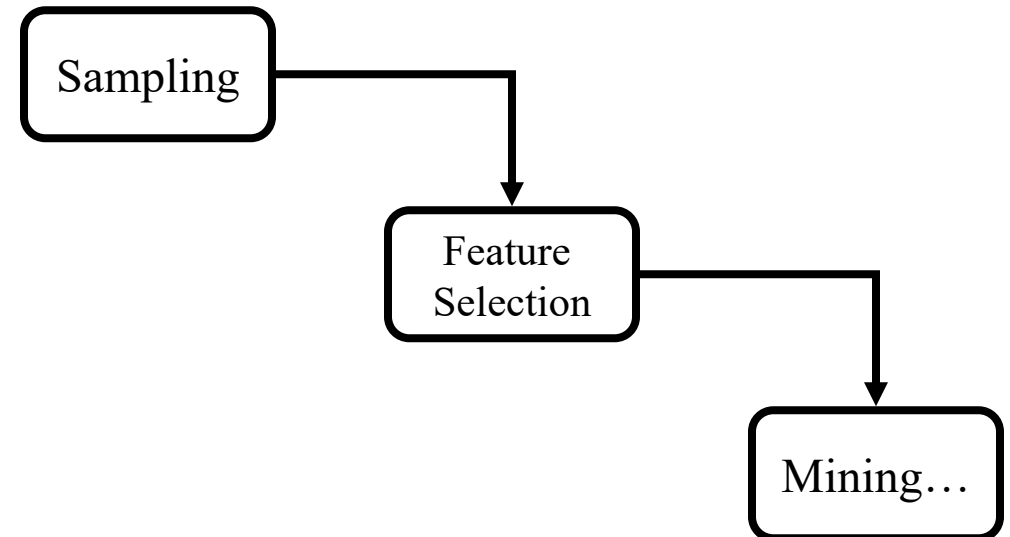
Un inconveniente: El preprocesamiento de datos no es un área totalmente estructurada con una metodología concreta de actuación para todos los problemas.

Cada problema puede requerir una actuación diferente, utilizando diferentes herramientas de preprocesamiento.

Comentarios finales

Un inconveniente: El preprocesamiento de datos no es un área totalmente estructurada con una metodología concreta de actuación para todos los problemas.

El diseño de procesos automáticos de uso de las diferentes etapas/técnicas en minería de datos es uno de los nuevos retos existentes.



Comentarios finales

Every problem can need a different preprocessing process, using different tools.



Knowledge-Based Systems

Volume 98, 15 April 2016, Pages 1–29

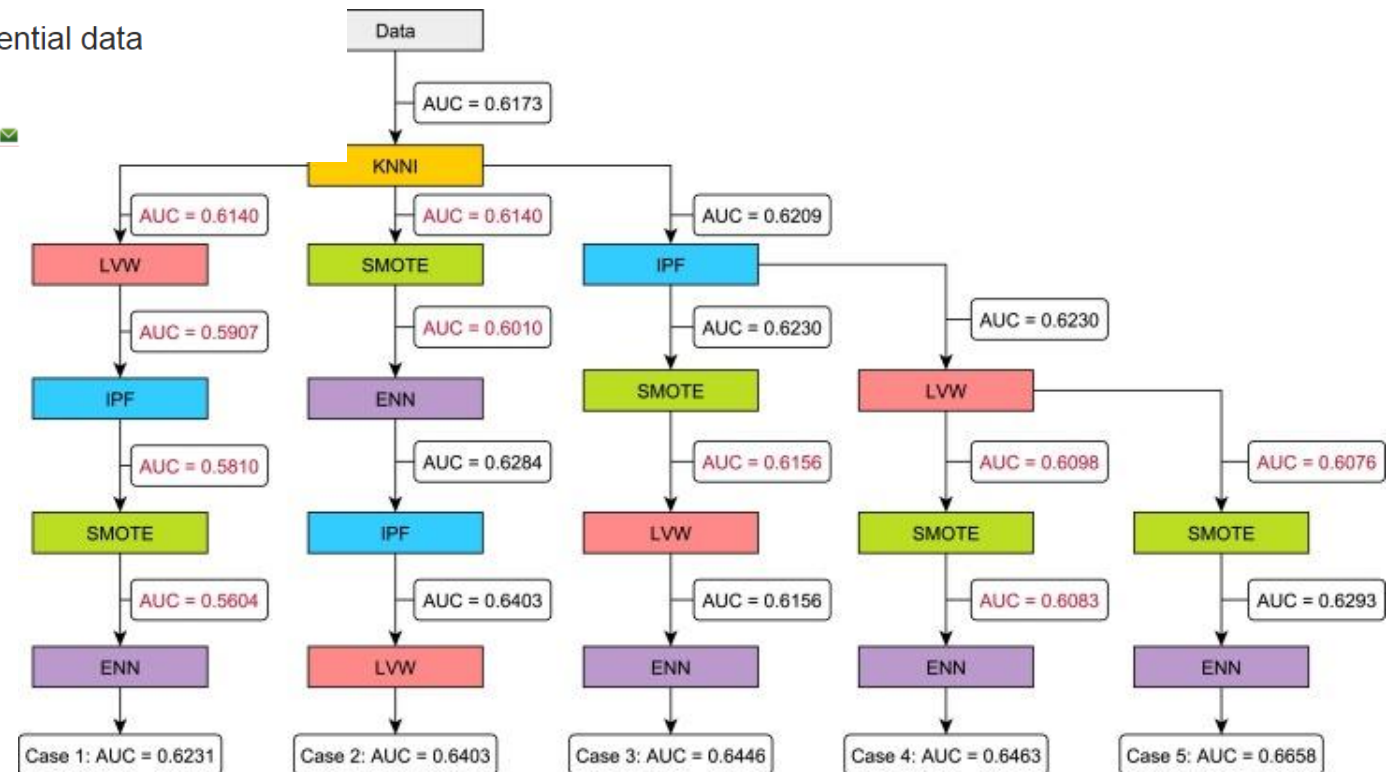


<http://sci2s.ugr.es/most-influential-preprocessing>

Tutorial on practical tips of the most influential data preprocessing algorithms in data mining

Salvador García^{a, b, *}, Julián Luengo^{a, *}, Francisco Herrera^{a, b, *}

Fig. 29. Depiction of the 5 Cases, hierarchically distributed by the preprocessing application order, showing the AUC values obtained in each step





Calidad de Datos

Calidad de datos

Introducción al preprocesamiento de datos

Integración, limpieza y transformación

Datos imperfectos

Reducción de datos

Discretización

Selección de Características

Selección de Instancias

Comentarios finales

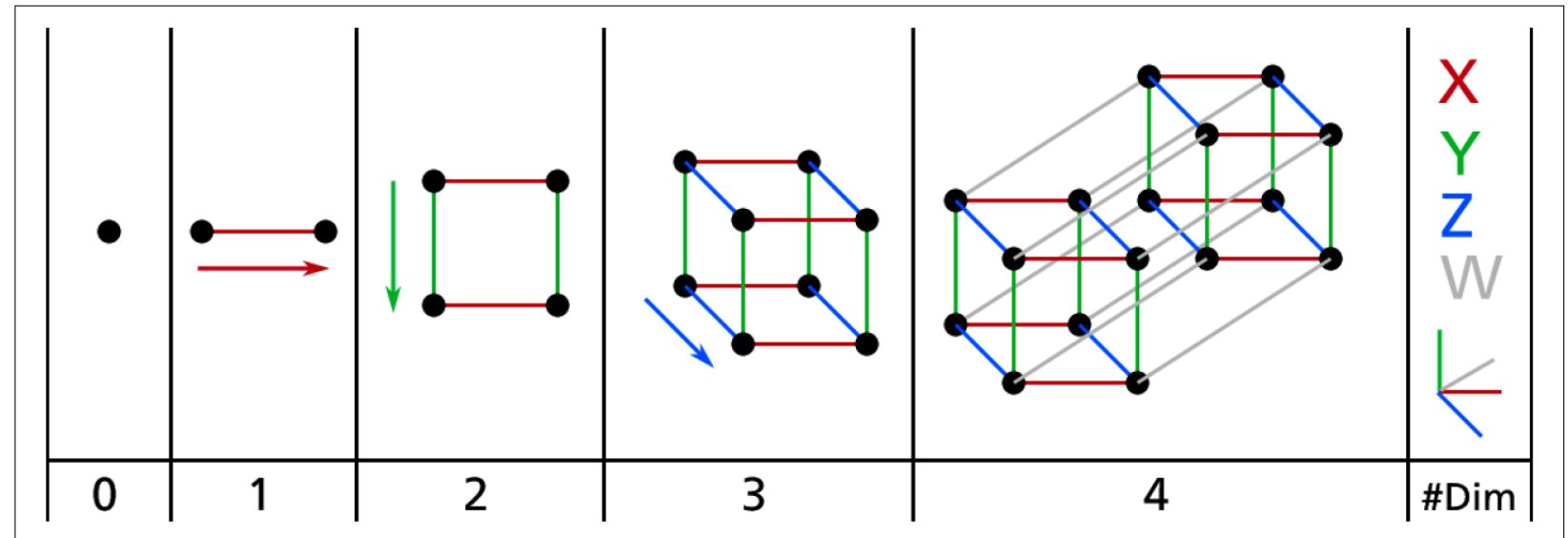
Suplemento: Reducción de Dimensionalidad

Reducción de la dimensionalidad

- Muchos problemas de aprendizaje automático implican miles o incluso millones de características para cada instancia de entrenamiento.
- Todas estas características no sólo hacen que el entrenamiento sea extremadamente lento, sino que también pueden hacer que sea mucho más difícil encontrar una buena solución.
- Este problema suele denominarse "*maldición de la dimensionalidad*".
- La reducción de la dimensionalidad también es muy útil para la visualización de datos.
- Reducir el número de dimensiones a **dos** permite representar en un gráfico una vista condensada de un conjunto de entrenamiento de altas dimensiones y, a menudo, obtener información importante mediante la detección visual de patrones, como los clústeres.

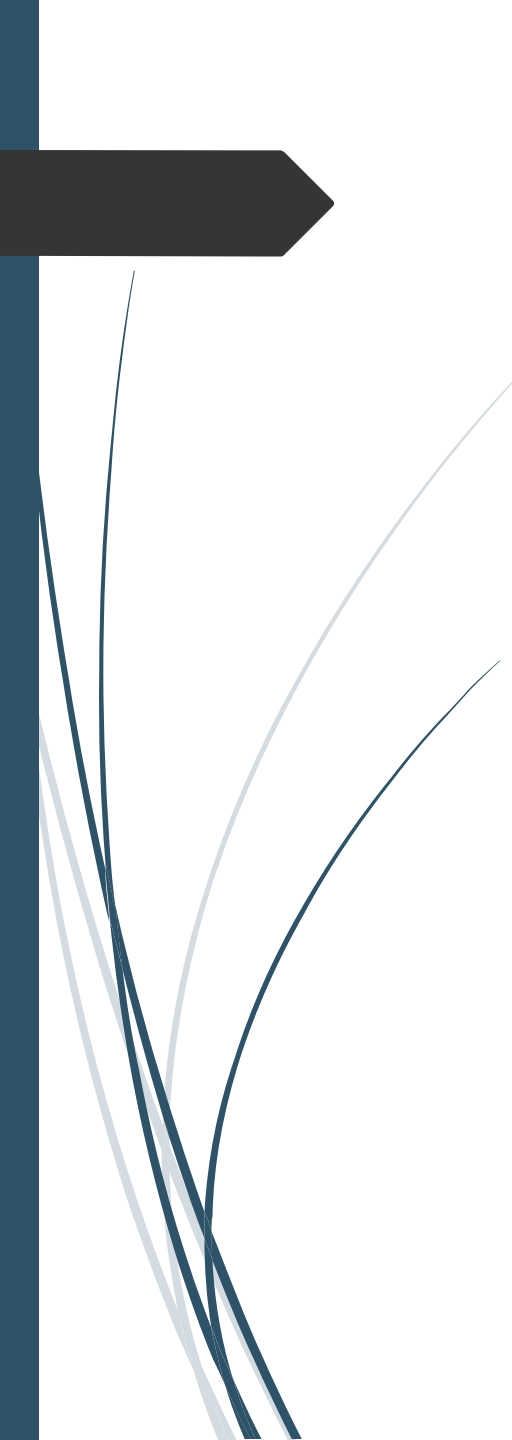
La maldición de la dimensionalidad

- Muchas cosas se comportan de manera muy diferente en el espacio de alta dimensión.
- Por ejemplo, si eliges un punto al azar en un cuadrado unitario (un cuadrado de 1×1), sólo tendrá un 0,4% de probabilidades de estar situado a menos de 0,001 de un borde. Pero en un hipercubo unitario de 10.000 dimensiones, esta probabilidad es superior al 99,999999%. La mayoría de los puntos de un hipercubo de alta dimensión están muy cerca de la frontera.



La maldición de la dimensionalidad

- En un hipercubo unitario de 1.000.000 de dimensiones, dos puntos aleatorios están muy separados. Simplemente hay mucho espacio en dimensiones altas.
- Como resultado, los conjuntos de datos de alta dimensión corren el riesgo de ser muy dispersos: es probable que la mayoría de las instancias de entrenamiento estén muy lejos unas de otras.
- Esto también significa que una nueva instancia probablemente estará muy lejos de cualquier instancia de entrenamiento, lo que hace que las predicciones sean mucho menos fiables que en dimensiones más bajas, ya que se basarán en extrapolaciones mucho mayores.
- En resumen, cuantas más dimensiones tenga el conjunto de entrenamiento, mayor será el riesgo de sobreajuste.
- Una solución a la maldición de la dimensionalidad podría ser aumentar el tamaño del conjunto de entrenamiento para alcanzar una densidad suficiente de instancias de entrenamiento.
- Por desgracia, el número de instancias de entrenamiento necesarias para alcanzar una densidad dada crece exponencialmente con el número de dimensiones.
- Con sólo 100 características (muchas menos que en el problema MNIST, que tiene 784), todas ellas comprendidas entre 0 y 1, se necesitarían más instancias de entrenamiento que átomos en el universo observable para que las instancias de entrenamiento tuvieran una diferencia media de 0,1 entre sí, suponiendo que estuvieran repartidas uniformemente por todas las dimensiones.

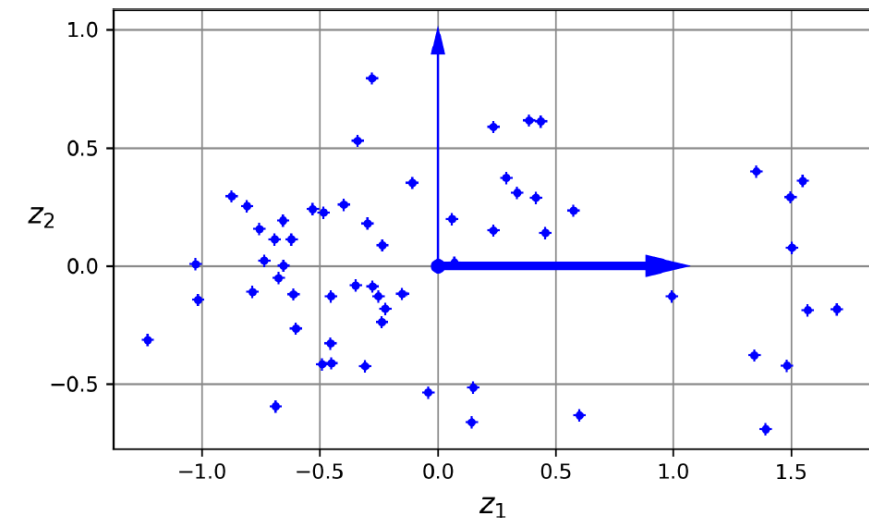
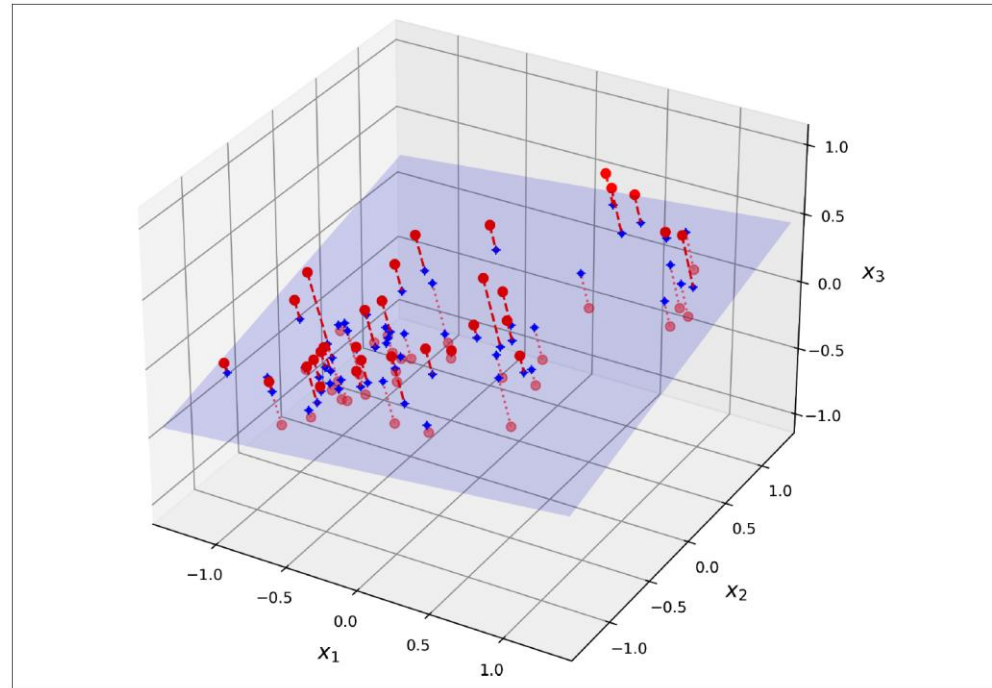


Proyección

- En la mayoría de los problemas del mundo real, las instancias de entrenamiento no están repartidas uniformemente por todas las dimensiones. Muchas características son casi constantes, mientras que otras están muy correlacionadas.
- Como resultado, todas las instancias de entrenamiento se encuentran dentro (o cerca) de un subespacio de dimensiones mucho más bajas del espacio de altas dimensiones.

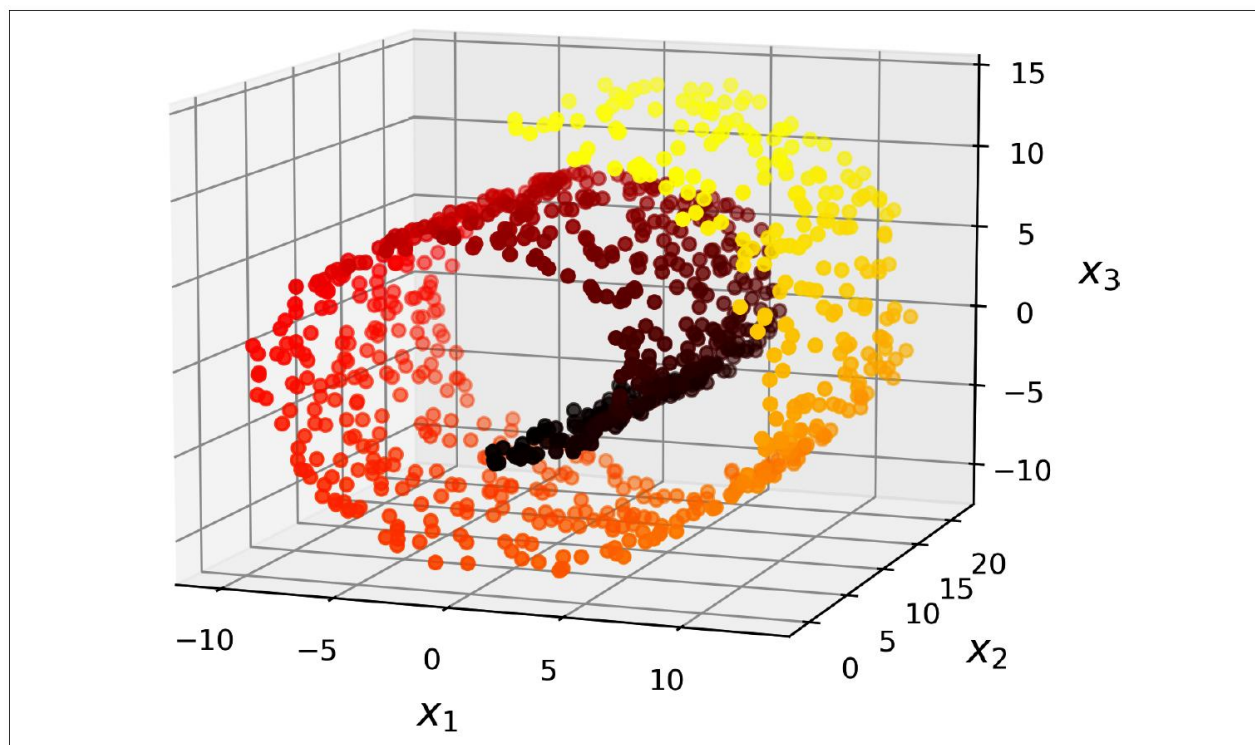
Proyección

- Aquí puedes ver un conjunto de datos 3D representado por pequeñas esferas.
- Se trata de un subespacio bidimensional del espacio tridimensional. Si proyectamos cada instancia de entrenamiento perpendicularmente sobre este subespacio (representado por las líneas discontinuas cortas que conectan las instancias con el plano), obtendremos el nuevo conjunto de datos 2D.



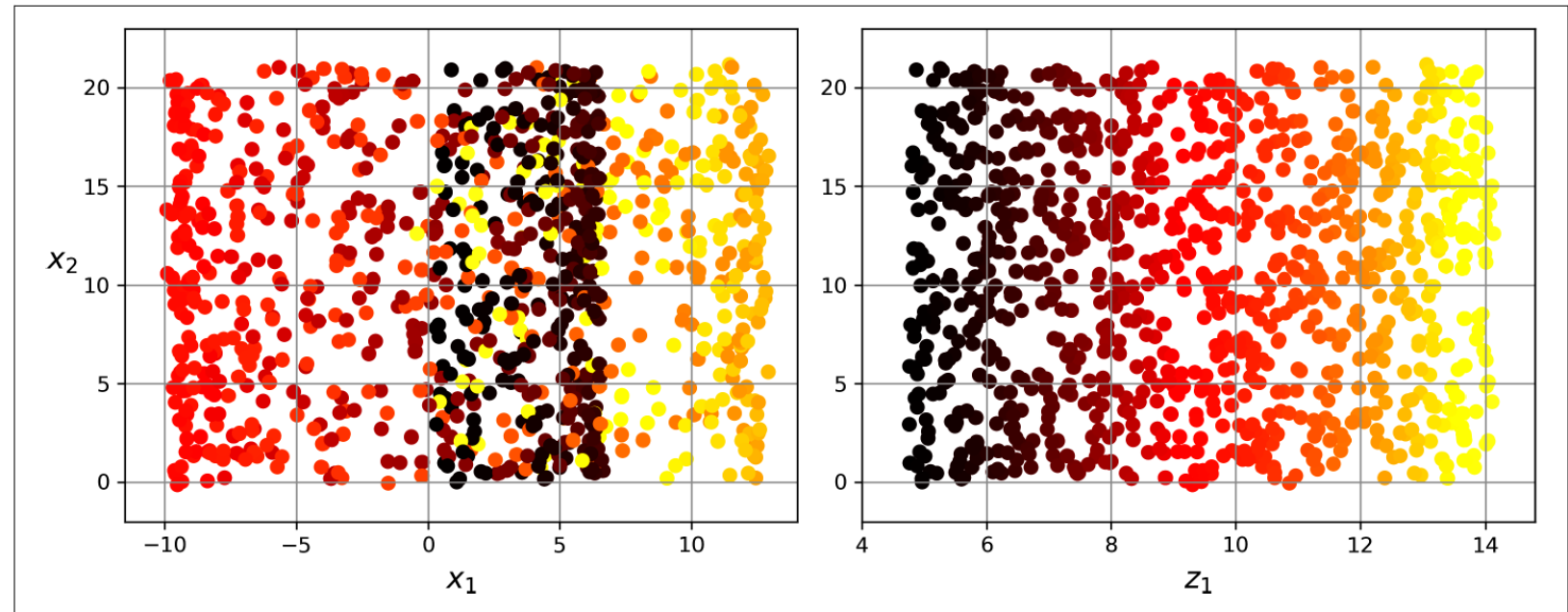
Aprendizaje múltiple (Manifold Learning)

- La proyección no siempre es el mejor enfoque para reducir la dimensionalidad. En muchos casos, el subespacio puede torcerse y girar, como en el famoso conjunto de datos de juguete de *rollo suizo* representado aquí:



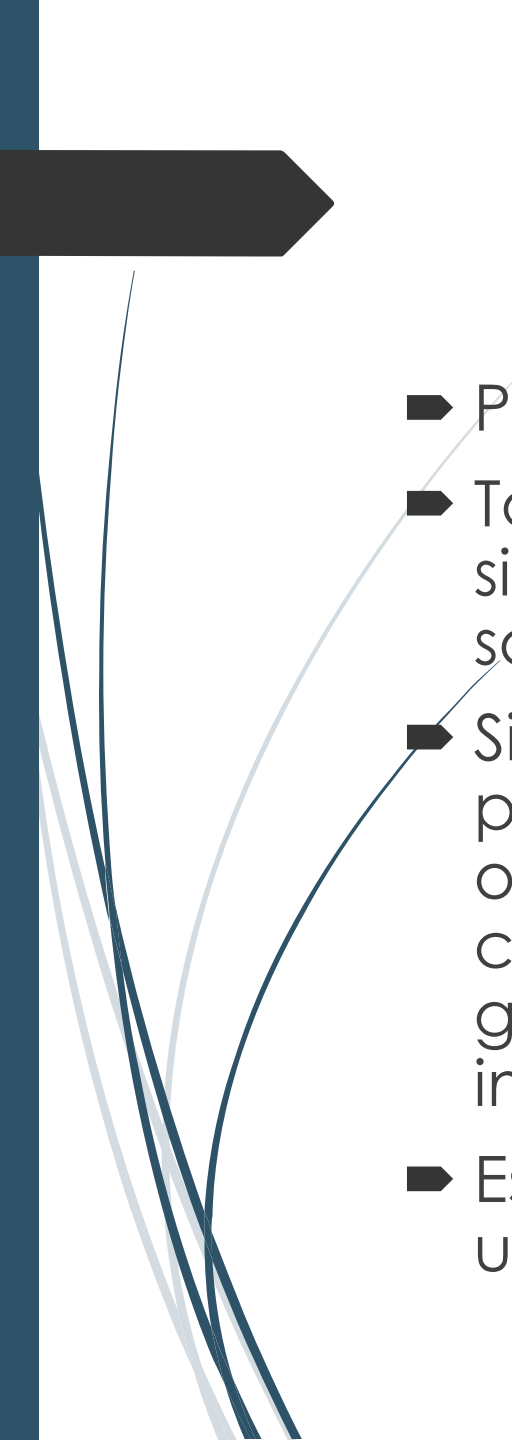
Aprendizaje múltiple

- La simple proyección sobre un plano (por ejemplo, dejando caer x_3) aplastaría diferentes capas del rollo suizo, como se muestra en la parte izquierda de esta figura.
- Lo que probablemente quieras en su lugar es desenrollar el rollo suizo para obtener el conjunto de datos 2D de la parte derecha de la figura.



Aprendizaje múltiple

- El rollo suizo es un ejemplo de colector (manifold) 2D, que es una forma 2D que puede doblarse y retorcerse en un espacio de dimensiones superiores.
- En términos más generales, un colector d -dimensional es una parte de un espacio n -dimensional (donde $d < n$) que localmente se asemeja a un hiperplano d -dimensional. En el caso del rollo suizo, $d = 2$ y $n = 3$: localmente se parece a un plano d -dimensional, pero se retuerce en la tercera dimensión.
- Muchos algoritmos de reducción de la dimensionalidad trabajan modelando la variedad en la que se encuentran las instancias de entrenamiento; esto se denomina *aprendizaje múltiple (manifold learning)*.
- Se basa en la *hipótesis de la multiplicidad*, que sostiene que la mayoría de los conjuntos de datos de alta dimensión del mundo real se aproximan a una multiplicidad de dimensiones mucho menores.
- Este supuesto se cumple empíricamente con mucha frecuencia.



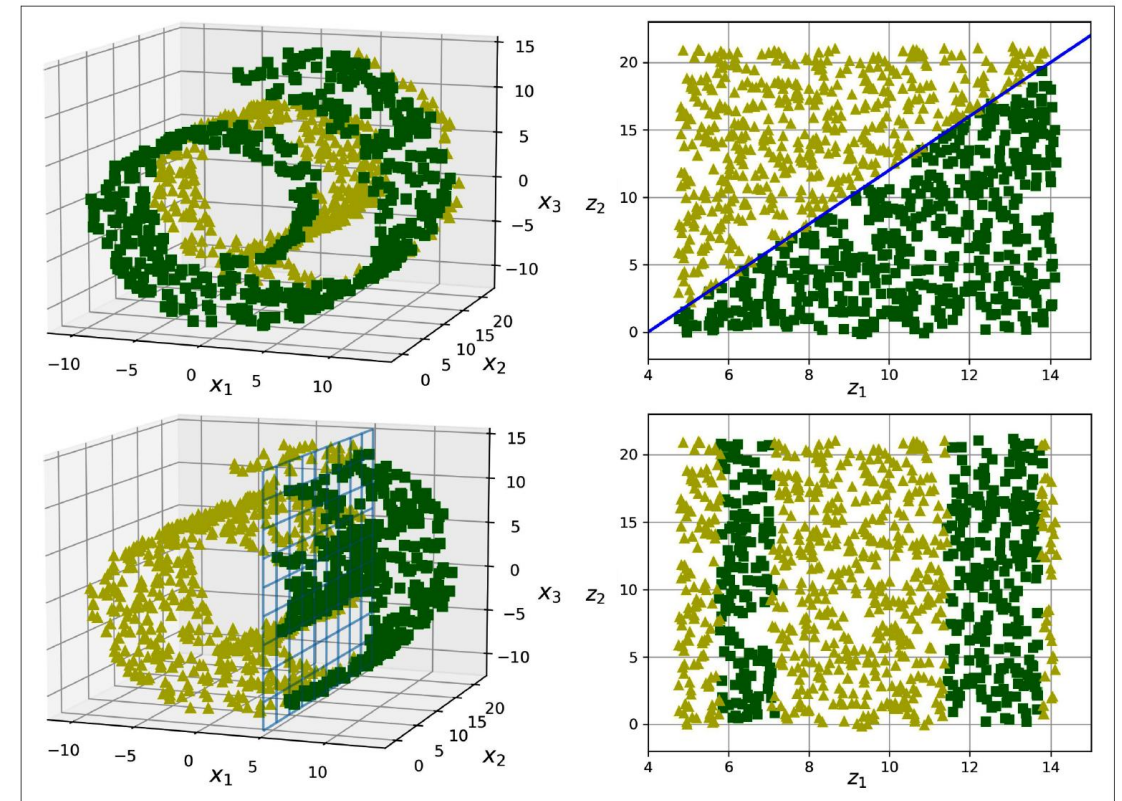
Aprendizaje múltiple

- Pensemos en el conjunto de datos MNIST.
- Todas las imágenes de dígitos manuscritos tienen algunas similitudes. Están formadas por líneas conectadas, los bordes son blancos y están más o menos centradas.
- Si generaras imágenes al azar, sólo una fracción ridículamente pequeña de ellas se parecerían a dígitos escritos a mano. En otras palabras, los grados de libertad de que dispone si intenta crear una imagen de dígitos son drásticamente inferiores a los grados de libertad que tiene si se le permite generar cualquier imagen que desee.
- Estas restricciones tienden a comprimir el conjunto de datos en una variedad de dimensiones inferiores.

Aprendizaje múltiple

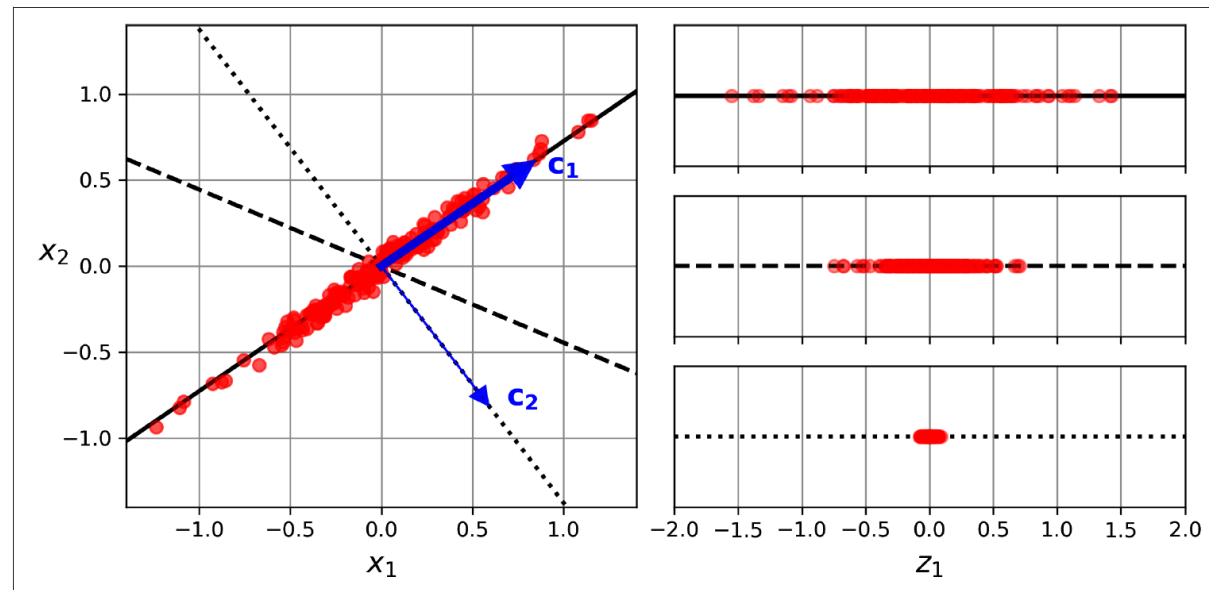
- La suposición de multiplicidad suele ir acompañada de otra suposición implícita: que la tarea en cuestión (por ejemplo, clasificación o regresión) será más sencilla si se expresa en el espacio de dimensiones inferiores del colector.

Reducir la dimensionalidad del conjunto de entrenamiento antes de entrenar un modelo suele acelerar el entrenamiento, pero no siempre conduce a una solución mejor o más sencilla; todo depende del conjunto de datos.



PCA. Preservación de la varianza

- El análisis de componentes principales (PCA) es, con diferencia, el algoritmo de reducción de la dimensionalidad más popular. En primer lugar, identifica el hiperplano más cercano a los datos y, a continuación, los proyecta sobre él.
- Antes de poder proyectar el conjunto de entrenamiento en un hiperplano de dimensiones inferiores, primero hay que elegir el hiperplano adecuado.
- Por ejemplo, en la siguiente figura se representa a la izquierda un conjunto de datos 2D simple, junto con tres ejes diferentes (es decir, hiperplanos 1D).



PCA. Componentes principales

- PCA identifica el eje que representa la mayor cantidad de varianza en el conjunto de entrenamiento.
- En la figura anterior, es la línea continua.
- También encuentra un segundo eje, ortogonal al primero, que explica la mayor parte de la varianza restante.
- En este ejemplo 2D no hay elección: es la línea de puntos. Si se tratara de un conjunto de datos de mayor dimensión, PCA también encontraría un tercer eje, ortogonal a los dos ejes anteriores, y un cuarto, un quinto, y así sucesivamente: tantos ejes como dimensiones tenga el conjunto de datos.
- El eje i^{th} se denomina *componente principal* (PC) i^{th} de los datos. En la figura anterior, el primer PC es el eje en el que se encuentra el vector $\mathbf{c}_1$, y el segundo PC es el eje en el que se encuentra el vector $\mathbf{c}_2$.

PCA. Componentes principales

- Entonces, ¿cómo encontrar los componentes principales de un conjunto de entrenamiento?
- Por suerte, existe una técnica estándar de factorización de matrices llamada *descomposición de valores singulares* (SVD) que puede descomponer la matriz $\mathbf{X}$ del conjunto de entrenamiento en la multiplicación de tres matrices $\mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$, donde $\mathbf{V}$ contiene los vectores unitarios que definen todos los componentes principales que está buscando, como se muestra aquí:

$$\mathbf{V} = \begin{pmatrix} | & | & & | \\ \mathbf{c}_1 & \mathbf{c}_2 & \cdots & \mathbf{c}_n \\ | & | & & | \end{pmatrix}$$

PCA. Proyección a dimensiones d

- Una vez identificados todos los componentes principales, se puede reducir la dimensionalidad del conjunto de datos a d dimensiones proyectándolo sobre el hiperplano definido por los primeros d componentes principales.
- La selección de este hiperplano garantiza que la proyección preservará la mayor varianza posible.
- Para proyectar el conjunto de entrenamiento en el hiperplano y obtener un conjunto de datos reducido $\mathbf{X}_{d\text{-proj}}$ de dimensionalidad d , calcula la multiplicación de la matriz del conjunto de entrenamiento $\mathbf{X}$ por la matriz $\mathbf{W}_d$, definida como la matriz que contiene las primeras d columnas de $\mathbf{V}$, como se muestra en

$$\mathbf{X}_{d\text{-proj}} = \mathbf{X}\mathbf{W}_d$$

PCA para compresión

- Tras la reducción de la dimensionalidad, el conjunto de entrenamiento ocupa mucho menos espacio. Por ejemplo, tras aplicar el PCA al conjunto de datos MNIST y conservar el 95% de su varianza, nos quedamos con 154 características, en lugar de las 784 originales. Por tanto, el conjunto de datos tiene ahora menos del 20% de su tamaño original, ¡y sólo hemos perdido el 5% de su varianza! Se trata de una relación de compresión razonable, y es fácil ver cómo una reducción de tamaño de este tipo aceleraría enormemente un algoritmo de clasificación.
- También es posible descomprimir el conjunto de datos reducido a 784 dimensiones aplicando la transformación inversa de la proyección PCA. Esto no le devolverá los datos originales, ya que la proyección perdió un poco de información, pero probablemente se acercará a los datos originales.
- La distancia media al cuadrado entre los datos originales y los reconstruidos (comprimidos y luego descomprimidos) se denomina *error de reconstrucción*.

$$\mathbf{X}_{\text{recovered}} = \mathbf{X}_{d\text{-proj}} \mathbf{W}_d^T$$

PCA incremental

- Uno de los problemas de las implementaciones originales del PCA es que requieren que todo el conjunto de entrenamiento quepa en la memoria para que el algoritmo pueda ejecutarse.
- Afortunadamente, se han desarrollado algoritmos *PCA incrementales* (IPCA) que permiten dividir el conjunto de entrenamiento en minilotes y alimentarlos de uno en uno.
- Esto es útil para grandes conjuntos de entrenamiento y para aplicar PCA en línea (es decir, sobre la marcha, a medida que llegan nuevas instancias).
- Para conjuntos de datos de dimensiones muy altas, el PCA puede ser demasiado lento. Incluso si utiliza PCA aleatorio, su complejidad computacional sigue siendo $O(m \times d^2) + O(d^3)$, por lo que el número objetivo de dimensiones d no debe ser demasiado grande. Si se trata de un conjunto de datos con decenas de miles de características o más (por ejemplo, imágenes), el entrenamiento puede resultar demasiado lento: en este caso, debería considerar utilizar la proyección aleatoria en su lugar.

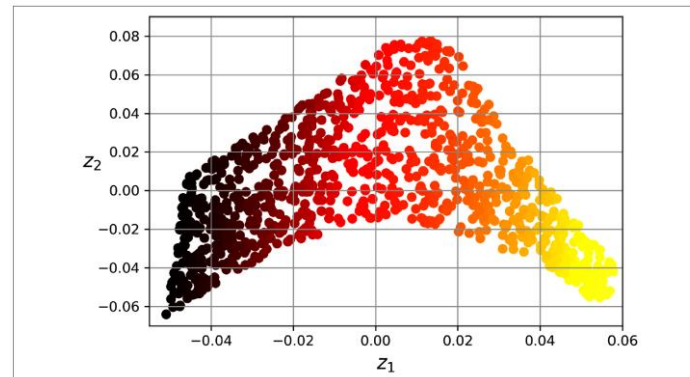


Proyección aleatoria

- El algoritmo de proyección aleatoria proyecta los datos a un espacio de dimensiones inferiores utilizando una proyección lineal aleatoria. Puede parecer una locura, pero resulta que es muy probable que una proyección aleatoria de este tipo conserve bastante bien las distancias. Así, dos casos similares seguirán siendo similares después de la proyección, y dos casos muy diferentes seguirán siendo muy diferentes.
- Obviamente, cuantas más dimensiones se eliminan, más información se pierde y más se distorsionan las distancias. Entonces, ¿cómo elegir el número óptimo de dimensiones? Existe una ecuación que determina el número mínimo de dimensiones que hay que conservar para garantizar, con una alta probabilidad, que las distancias no cambien más allá de una tolerancia determinada.
- La proyección aleatoria es un algoritmo de reducción de la dimensionalidad sencillo, rápido, eficiente en memoria y sorprendentemente potente que debería tener en cuenta, especialmente cuando trabaja con conjuntos de datos de alta dimensión.

LLE

- La *incrustación lineal local* (LLE) es una técnica de *reducción no lineal de la dimensionalidad* (NLDR).
- Se trata de una técnica de aprendizaje múltiple que no se basa en proyecciones, a diferencia del PCA y la proyección aleatoria.
- La LLE mide primero cómo se relaciona linealmente cada instancia de entrenamiento con sus vecinos más próximos y, a continuación, busca una representación de baja dimensión del conjunto de entrenamiento en la que estas relaciones locales se conserven mejor.
- Este enfoque lo hace especialmente bueno para desenrollar colectores retorcidos, sobre todo cuando no hay demasiado ruido.



LLE

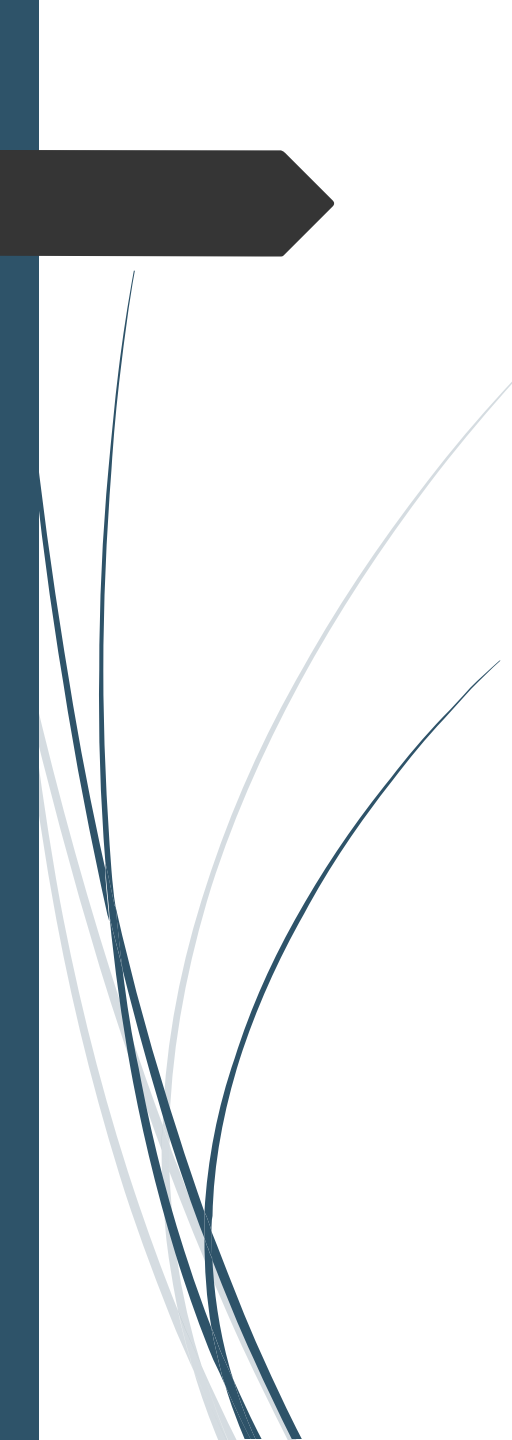
- Cómo funciona LLE:
- Para cada instancia de entrenamiento $\mathbf{x}^{(i)}$, el algoritmo identifica a sus k vecinos más cercanos y, a continuación, intenta reconstruir $\mathbf{x}^{(i)}$ como una función lineal de estos vecinos.
- Más concretamente, intenta encontrar las ponderaciones $w_{i,j}$ tales que la distancia al cuadrado entre $\mathbf{x}^{(i)}$ y $\sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)}$ sea lo más pequeña posible, asumiendo $w_{i,j} = 0$ si $\mathbf{x}^{(j)}$ no es uno de los k vecinos más cercanos de $\mathbf{x}^{(i)}$.
- El primer paso de LLE es el problema de optimización con restricciones descrito en la siguiente ecuación, donde $\mathbf{W}$ es la matriz de pesos que contiene todos los pesos $w_{i,j}$. La segunda restricción simplemente normaliza los pesos para cada instancia de entrenamiento $\mathbf{x}^{(i)}$.

$$\begin{aligned} \widehat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \quad & \sum_{i=1}^m \left(\mathbf{x}^{(i)} - \sum_{j=1}^m w_{i,j} \mathbf{x}^{(j)} \right)^2 \\ \text{subject to} \quad & \begin{cases} w_{i,j} = 0 & \text{if } \mathbf{x}^{(j)} \text{ is not one of the } k \text{ n.n. of } \mathbf{x}^{(i)} \\ \sum_{j=1}^m w_{i,j} = 1 & \text{for } i = 1, 2, \dots, m \end{cases} \end{aligned}$$

LLE

- Tras este paso, la matriz de pesos $\hat{\mathbf{W}}$ (que contiene los pesos $\hat{w}_{i,j}$) codifica las relaciones lineales locales entre las instancias de entrenamiento.
- El segundo paso consiste en mapear las instancias de entrenamiento en un espacio d -dimensional (donde $d < n$) preservando estas relaciones locales tanto como sea posible.
- Si $\mathbf{z}^{(i)}$ es la imagen de $\mathbf{x}^{(i)}$ en este espacio d -dimensional, entonces queremos que la distancia al cuadrado entre $\mathbf{z}^{(i)}$ y $\sum_{j=1}^m \hat{w}_{i,j} \mathbf{z}^{(j)}$ sea lo más pequeña posible.
- Esta idea conduce al problema de optimización sin restricciones descrito en la siguiente ecuación. Es muy similar al primer paso, pero en lugar de mantener las instancias fijas y encontrar los pesos óptimos, estamos haciendo lo contrario: mantener los pesos fijos y encontrar la posición óptima de las imágenes de las instancias en el espacio de baja dimensión. Obsérvese que $\mathbf{Z}$ es la matriz que contiene todas las $\mathbf{z}^{(i)}$.

$$\hat{\mathbf{Z}} = \underset{\mathbf{Z}}{\operatorname{argmin}} \sum_{i=1}^m \left(\mathbf{z}^{(i)} - \sum_{j=1}^m \hat{w}_{i,j} \mathbf{z}^{(j)} \right)^2$$



LLE

- La implementación LLE de Scikit-Learn tiene la siguiente complejidad computacional: $O(m \log(m)n \log(k))$ para encontrar los k vecinos más cercanos, $O(mnk^3)$ para optimizar los pesos, y $O(dm^2)$ para construir las representaciones de baja dimensión.
- Desgraciadamente, la m^2 del último término hace que este algoritmo se adapte mal a conjuntos de datos muy grandes.
- Como puede ver, LLE es bastante diferente de las técnicas de proyección, y es significativamente más complejo, pero también puede construir representaciones de baja dimensión mucho mejores, especialmente si los datos son no lineales.



Otras técnicas de reducción de la dimensionalidad

► MDS

- *El escalado multidimensional (MDS) reduce la dimensionalidad al tiempo que intenta preservar las distancias entre las instancias. La proyección aleatoria lo hace para datos de alta dimensión, pero no funciona bien en datos de baja dimensión.*

► Isomap

- *Isomap crea un grafo conectando cada instancia a sus vecinos más cercanos y, a continuación, reduce la dimensionalidad al tiempo que intenta preservar las distancias geodésicas entre las instancias. La distancia geodésica entre dos nodos de un grafo es el número de nodos del camino más corto entre esos nodos.*

► t-SNE

- *La incrustación estocástica de vecinos t -distribuida (t-SNE) reduce la dimensionalidad a la vez que intenta mantener las instancias similares cerca y las instancias disímiles separadas. Se utiliza sobre todo para la visualización, en particular para visualizar grupos de instancias en un espacio de alta dimensión.*

Otras técnicas de reducción de la dimensionalidad

► LDA

- El análisis lineal discriminante (LDA) es un algoritmo de clasificación lineal que, durante el entrenamiento, aprende los ejes más discriminantes entre las clases. Estos ejes pueden utilizarse para definir un hiperplano sobre el que proyectar los datos. La ventaja de este enfoque es que la proyección mantendrá las clases lo más separadas posible, por lo que el LDA es una buena técnica para reducir la dimensionalidad antes de ejecutar otro algoritmo de clasificación.

